

学习内容

CONTENTS



1

神经元与神经网络

2

BP神经网络

3

卷积神经网络

4

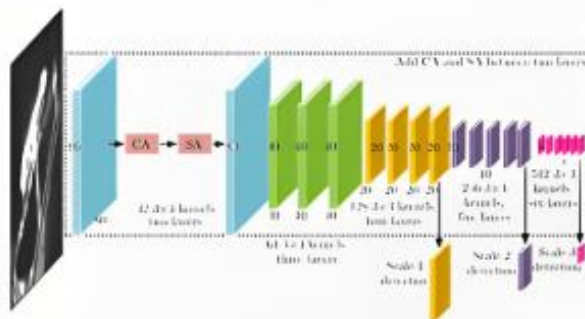
生成对抗网络

5

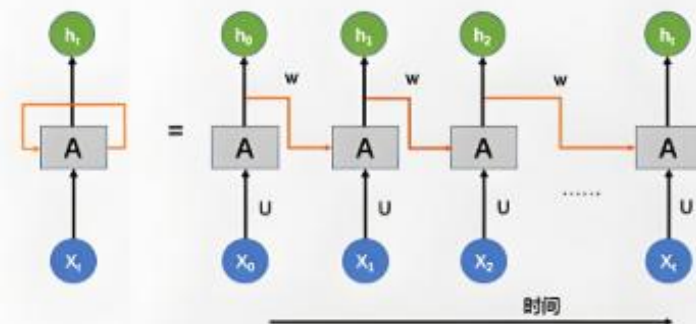
循环神经网络

6

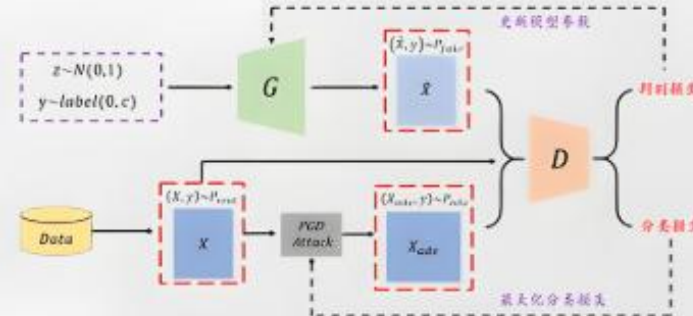
Transformer



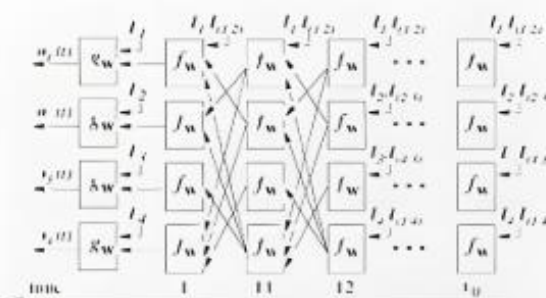
CNN卷积



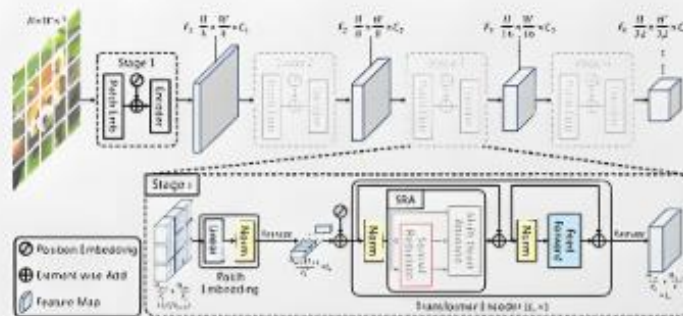
RNN循环



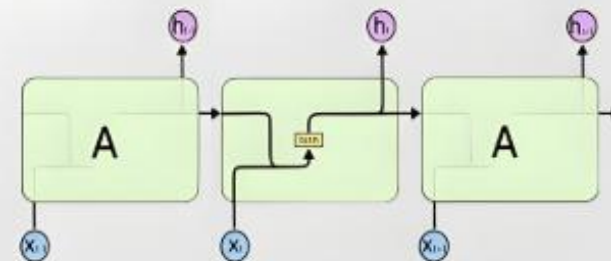
GAN生成



GNN图神经



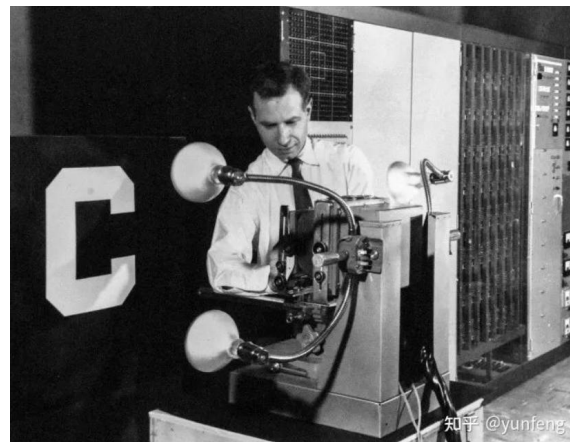
Transformer



The repeating module in a standard RNN contains a single layer.

LSTM长短期

1957年，Frank Rosenblatt成功在IBM 704机上完成了感知机的仿真，1958年发表，1959年又成功实现了能够识别一些英文字母、**基于感知机的神经计算机——Mark1**，并于1960年6月23日，展示与众。



Frank Rosenblatt “Mark 1 感知机”

FIG. 1 — Organization of a biological brain. (Red areas indicate active cells, responding to the letter X.)

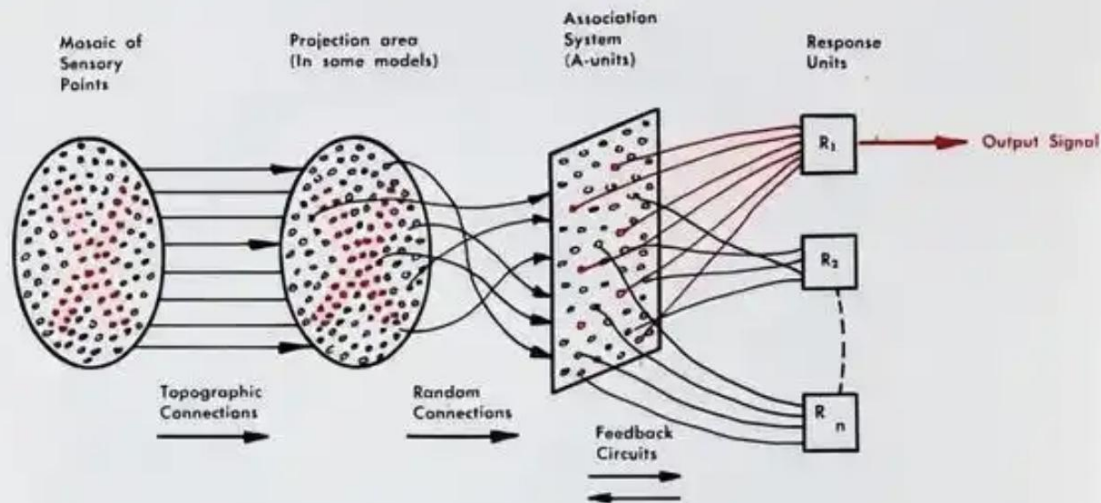
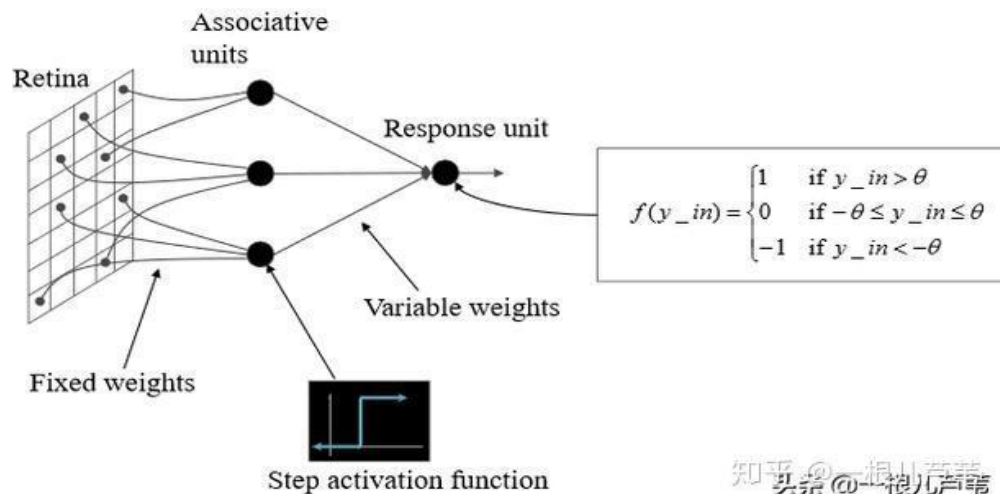


FIG. 2 — Organization of a perceptron.

CSDN @KarenMiao

感知机(Perceptrons)

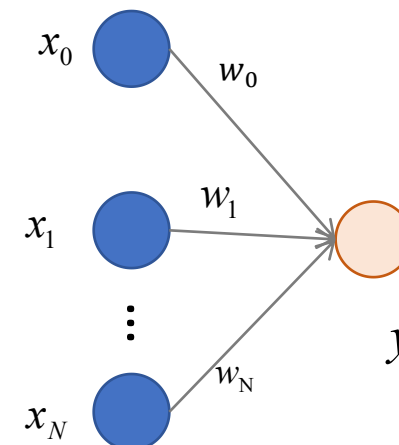
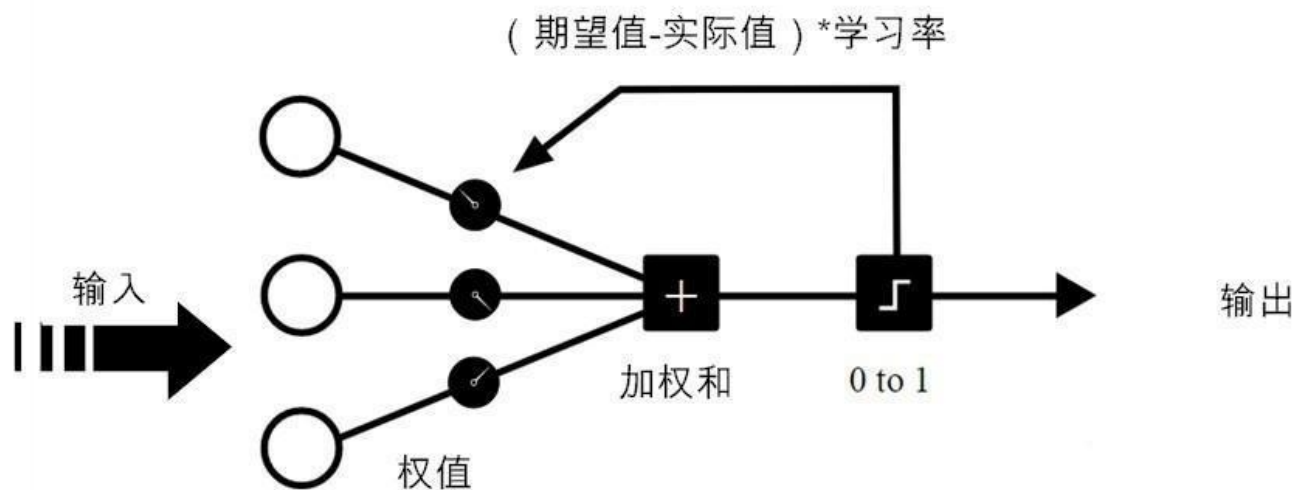


知乎 @一根儿芦苇

A Perceptron

结合M-P模型和Hebb学习规则，解决了M-P模型中参数W的学习问题

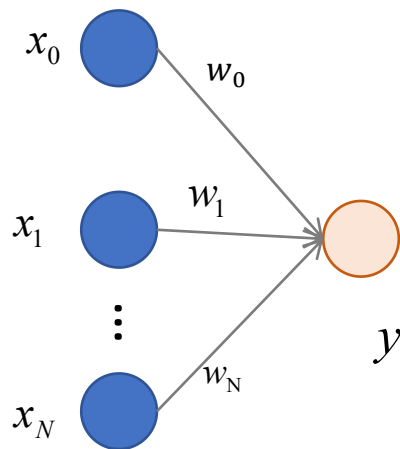
神经网络的“Hello World”：单层感知器模型



输入神经元 输出神经元

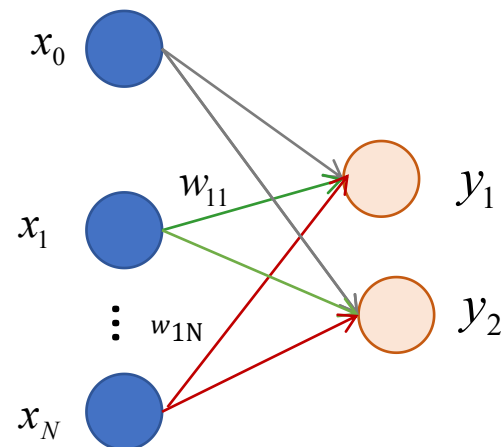
$$y = f(a) = f\left(\sum_{i=0}^N w_i x_i\right) = \begin{cases} 1 & a \geq 0 \\ 0 & a < 0 \end{cases}$$

神经网络的“Hello World”：单层感知器模型



输入神经元 输出神经元

$$y = f(a) = f\left(\sum_{i=0}^N w_i x_i\right) = \begin{cases} 1 & a \geq 0 \\ 0 & a < 0 \end{cases}$$



输入神经元 输出神经元

$$y_i = f(a_i) = f\left(\sum_{j=0}^N w_{ij} x_j\right) = \begin{cases} 1 & a_i \geq 0 \\ 0 & a_i < 0 \end{cases}$$

$$i = 1, 2$$

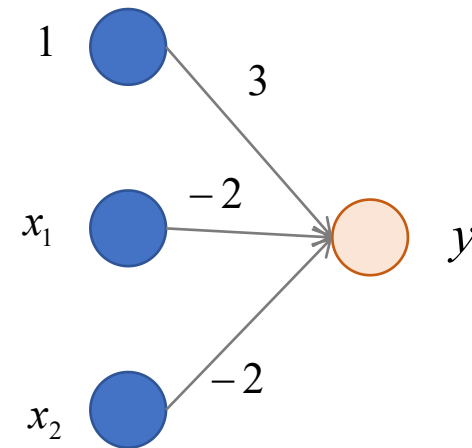
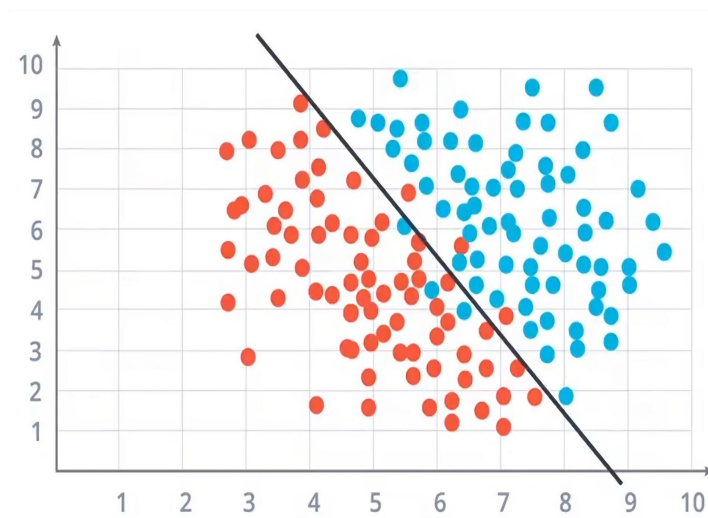
单层感知器模型可实现线性分类

示例：与非门

$$y = f(a) = f\left(\sum_{i=0}^N w_i x_i\right) = \begin{cases} 1 & a \geq 0 \\ 0 & a < 0 \end{cases}$$

		x2	
		0	1
x1	0	1	1
	1	1	0

单层感知机只能解决线性可分问题
Marvin Lee Minsky 和 Seymour Papert, 《Perceptrons》, 1969 年



x_1	x_2	a	y
0	1	1	1
1	0	1	1
0	0	3	1
1	1	-1	0

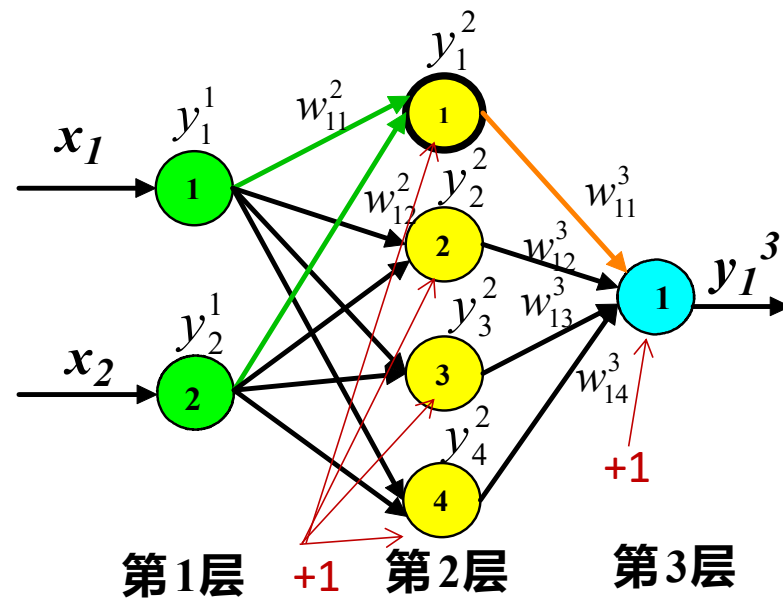
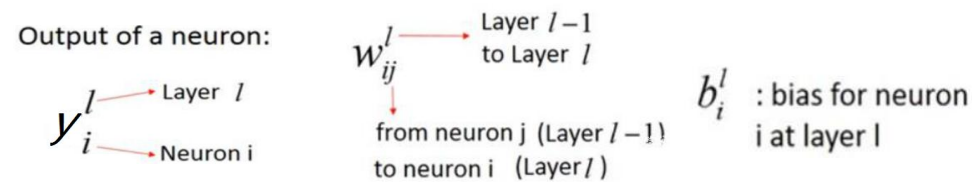
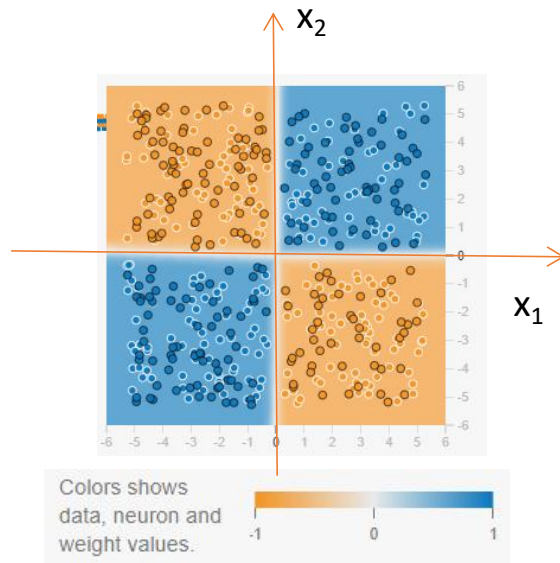
8.2

多层感知器 (Multi-Layer Perceptron)

单层感知器模型难以解决线性不可分的问题

示例：异或门

		x2	
		0	1
x1	0	0	1
	1	1	0



- 第1层（输入层）神经元输出

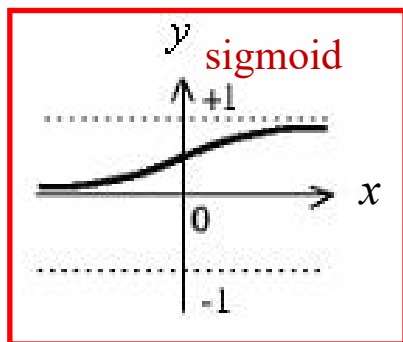
$$y_1^1 = x_1$$

$$y_2^1 = x_2$$

- 第2层（隐层）神经元输出

$$u_i^2 = \sum_{j=1}^2 w_{ij}^2 y_j^1 + b_i^2$$

$$y_i^2 = f(u_i^2) = \frac{1}{1 + e^{-u_i^2}} \quad i = 1, 2, \dots, 4$$



- 第3层（输出层）神经元输出

$$u_1^3 = w_{11}^3 y_1^2 + w_{12}^3 y_2^2 + w_{13}^3 y_3^2 + w_{14}^3 y_4^2 + b_1^3 = \sum_{j=1}^4 w_{1j}^3 y_j^2 + b_1^3$$

$$y_1^3 = f(u_1^3) = \frac{1}{1 + e^{-u_1^3}}$$

◆ 输入输出变换关系 (2- 4- 1)

$$u_i^k = \sum_{j=1}^{p_{k-1}} w_{ij}^k y_j^{k-1} + b_i^k \quad i=1,2,\dots,p_k$$

$$y_i^k = f(u_i^k) \quad k=2, 3$$

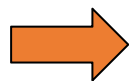


$$\begin{bmatrix} u_1^k \\ u_2^k \\ \vdots \\ u_i^k \\ \vdots \end{bmatrix}_{p_k} = \begin{bmatrix} w_{11}^k & w_{12}^k & \dots \\ w_{21}^k & w_{22}^k & \\ \vdots & & \ddots \end{bmatrix} \begin{bmatrix} y_1^{k-1} \\ y_2^{k-1} \\ \vdots \\ y_i^{k-1} \\ \vdots \end{bmatrix}_{p_{k-1}} + \begin{bmatrix} b_1^k \\ b_2^k \\ \vdots \\ b_i^k \\ \vdots \end{bmatrix}$$



$$u^k = W^k y^{k-1} + b^k$$

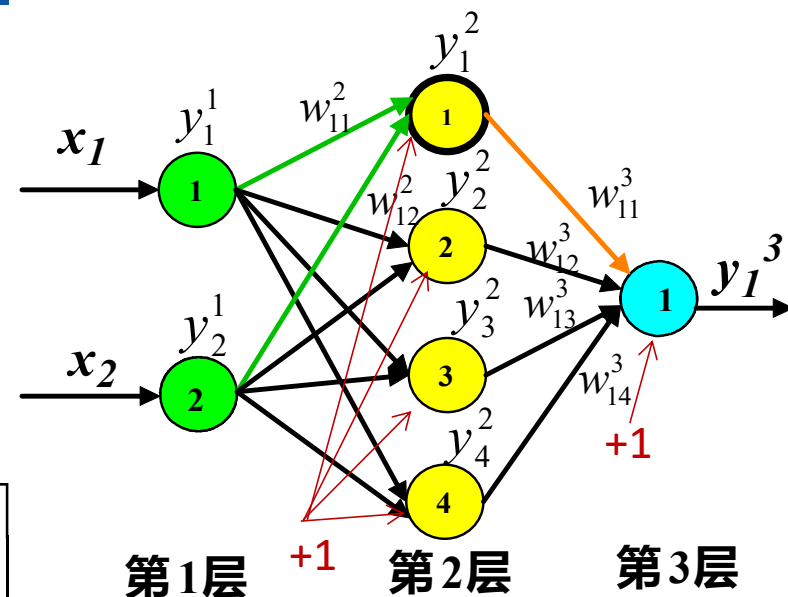
$$y^k = f^k(u^k) \quad k=2, 3$$



两层BP神经网络模型的一般表达式:

$$y^k = f^k(W^k y^{k-1} + b^k), \quad k=2,3$$

$$y^k = f^k([y^{k-1}]^T [W^k]^T + b^k), \quad k=2,3$$

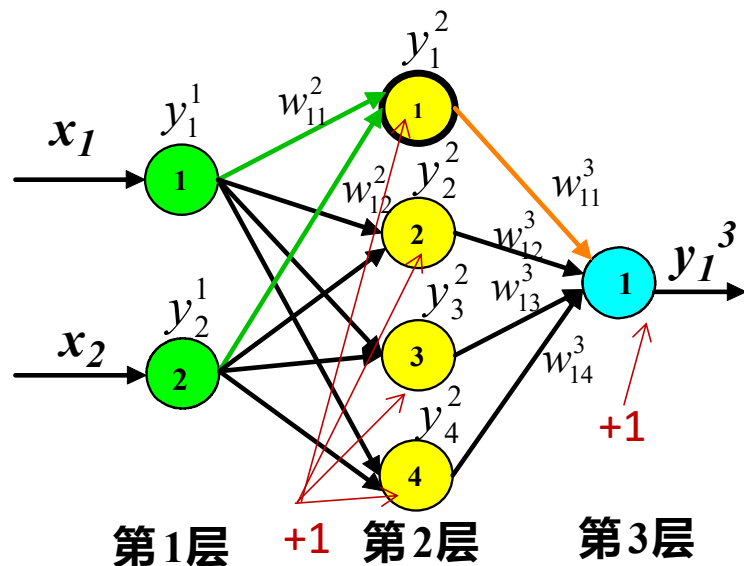


◆ 输入输出变换关系 (2- 4- 1)

$$y^k = f^k(W^k y^{k-1} + b^k), \quad k = 2, 3$$

$$y^k = f^k([y^{k-1}]^T [W^k]^T + b^k), \quad k = 2, 3$$

x_1	x_2	y
0	0	0
1	0	1
0	1	1
1	1	0



#张量方式实现

```
import torch as t
from torch import nn

x =
torch.tensor([[0,0],[1,0],[0,1],[1,1]])
# 创建W, b张量
w2 = nn.Parameter(t.randn(2, 4))
b2 = nn.Parameter(t.randn(4))
y2 = t.Sigmoid(x.mm(w2) + b2)

w3 = nn.Parameter(t.randn(4, 1))
b3 = nn.Parameter(t.randn(1))
y3 = t.Sigmoid(y2.mm(w3) + b3)
```

每一层输出 shape: (批量大小, 神经元个数/输出特征维度)

#层方式实现

```
fc1 = nn.Linear(2,4)
fc2 = nn.Linear(4,1)
y2 = t.Sigmoid(fc1(x))
y3 =
t.Sigmoid(fc2(y2))
```

#使用Sequential 容器

```
model =
nn.Sequential(
    nn.Linear(2,4),
    nn.Sigmoid(),
    nn.Linear(4,1),
    nn.Sigmoid())

y3 = model(x)
```

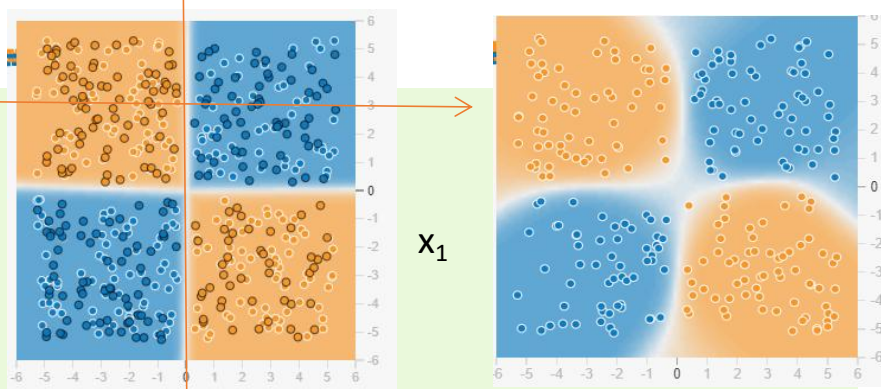
考虑权值矩阵 W , 偏置向量 b , 输入特征长度为 d_{in} , 输出特征长度为 d_{out} 的全连接层, 该层的参数量为 $d_{in} * d_{out} + d_{out}$. 该层的计算量 (浮点运算次数 FLOPS, 乘加运算次数) 为 $[d_{in} + d_{in} - 1 + 1] * d_{out} = 2d_{in}d_{out}$.

2-4-1全连接网络的总参数量?

$$2*4 + 4 + 4*1 + 1 = 17$$

示例：异或门

```
import torch.nn as nn
from thop import profile
mlp = nn.Sequential(
    nn.Linear(2, 4),
    nn.Sigmoid(),
    nn.Linear(4, 1),
    nn.Sigmoid()
)
print(mlp) # 打印模型结构
```

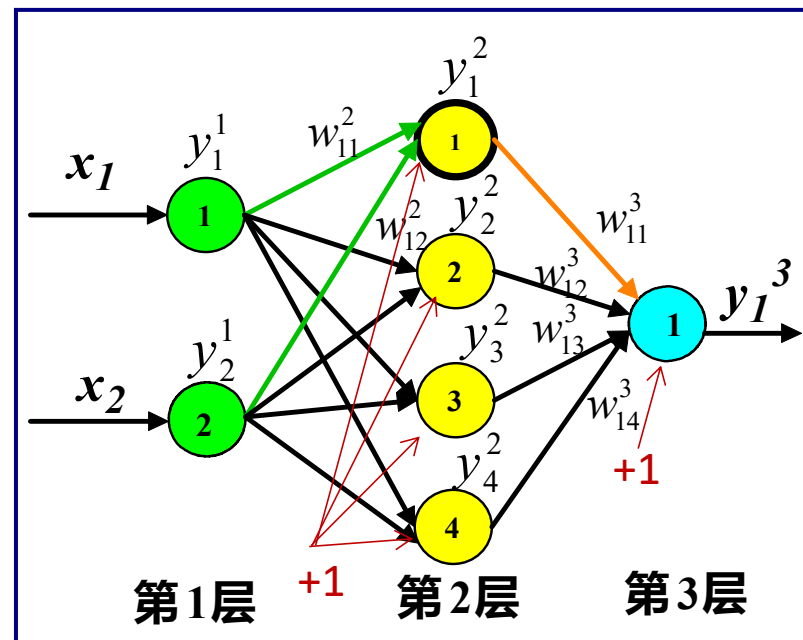


模型结构:

```
Sequential(
  (0): Linear(in_features=2, out_features=4, bias=True)
  (1): Sigmoid()
  (2): Linear(in_features=4, out_features=1, bias=True)
  (3): Sigmoid()
)
```

```
total_params = sum(param.numel() for param in mlp.parameters())
for name, param in mlp.named_parameters():
    print(f"{name}: 形状 {param.shape}, 参数数量: {param.numel()}")
print(f"\n总参数数量: {total_params}")
```

```
flops, total_params1 = profile(mlp, inputs=(torch.randn(1, 2),))
print(f"\nFLOPs: {flops:.2f}, 总参数数量: {total_params1}")
```



两层感知器模型的一般表达式:

$$y^k = f^k(W^k y^{k-1} + b^k), \quad k = 2, 3$$

隐藏层激活函数: Sigmoid

输出层激活函数: Sigmoid

损失函数: 二分类交叉熵

示例：异或门

```
import torch.nn as nn
from thop import profile
mlp = nn.Sequential(
    nn.Linear(2, 4),
    nn.Sigmoid(),
    nn.Linear(4, 1),
    nn.Sigmoid()
)
print(mlp) # 打印模型结构
```

```
total_params = sum(param.numel() for param in mlp.parameters())
for name, param in mlp.named_parameters():
    print(f"{name}: 形状 {param.shape}, 参数数量: {param.numel()}")
print(f"\n总参数数量: {total_params}")
```

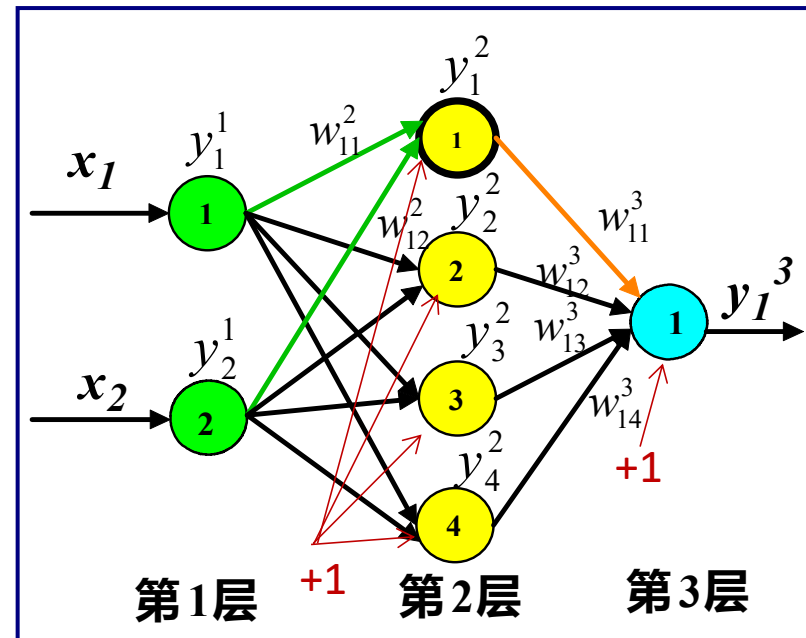
```
flops, total_params1 = profile(mlp, inputs=(torch.randn(1, 2),))
print(f"\nFLOPs: {flops:.2f}, 总参数数量: {total_params1}")
```

两层感知器模型的一般表达式:

$$y^k = f^k(W^k y^{k-1} + b^k), \quad k = 2, 3$$

模型结构:

```
Sequential(
  (0): Linear(in_features=2, out_features=4, bias=True)
  (1): Sigmoid()
  (2): Linear(in_features=4, out_features=1, bias=True)
  (3): Sigmoid())
```

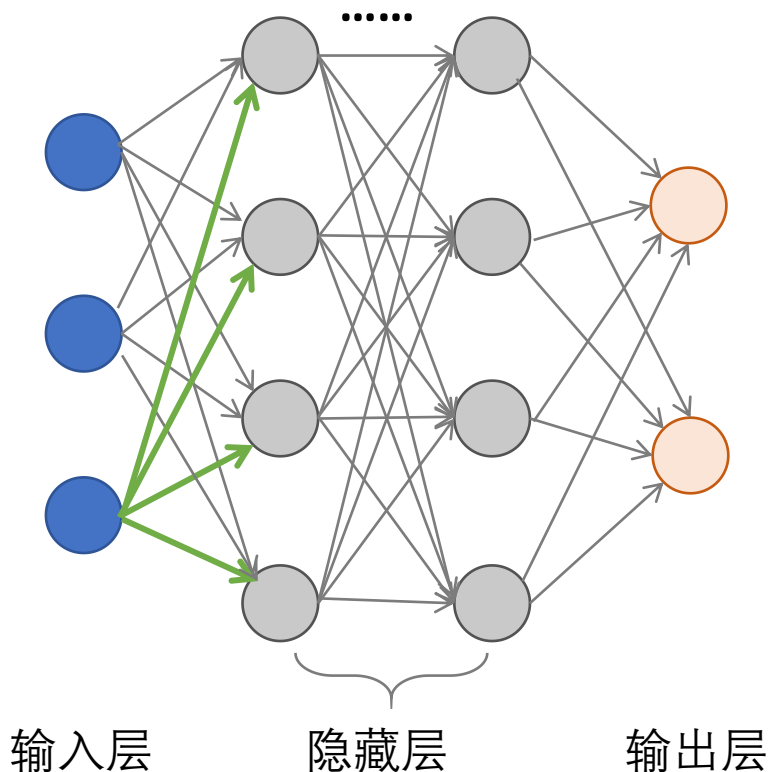


参数信息:

```
0.weight: 形状 torch.Size([4, 2]), 参数数量: 8
0.bias: 形状 torch.Size([4]), 参数数量: 4
2.weight: 形状 torch.Size([1, 4]), 参数数量: 4
2.bias: 形状 torch.Size([1]), 参数数量: 1
```

总参数数量: 17

FLOPs: 12.00, 总参数数量: 17.0



输入层：接收向量 x 的输入，输入层的神经元数量取决于向量 x 的长度。

隐藏层：介于输入层与输出层之间，执行三个动作（加权求和、加偏置、经过激活函数）

输出层：输出预测结果 y ， y 可以是一个值，也可以是由多个值组成的向量。

$$y^k = f^k(W^k y^{k-1} + b^k), \quad k = 2, \dots, M$$

第 k 层的输出



David Rumelhart

- Rumelhart, Hinton和 Williams于1986年提出了**误差反向传播BP(Back Propagation)**学习算法，解决了多层感知器的训练问题。
- BP神经网络是一种按**误差反向传播算法**训练的多层前馈网络**MLP**，应用最广泛的神经网络模型之一。
- BP算法的核心是使用**最速下降法**，通过**反向传播**来不断调整网络参数，使网络的**误差平方和最小**。

- 特征值：面积、楼层、户型...
- 标签：价格

1. 工作过程

• 第一阶段或称网络训练阶段：

■ 有 n 个训练样本，其中第 k 个样本：

特征值： $\mathbf{x}^{(k)} = [x_1^{(k)}, x_2^{(k)}, \dots, x_m^{(k)}]$

标签： $y^{(k)} = [y_1^{(k)}, y_2^{(k)}, \dots, y_p^{(k)}]$

$$k = 1, 2, \dots, n$$

$$\left\{ \begin{array}{l} (x_1^{(1)}, x_2^{(1)}, x_3^{(1)}, x_4^{(1)}, x_5^{(1)}, y^{(1)}) \\ (x_1^{(2)}, x_2^{(2)}, x_3^{(2)}, x_4^{(2)}, x_5^{(2)}, y^{(2)}) \\ \dots \\ (x_1^{(n)}, x_2^{(n)}, x_3^{(n)}, x_4^{(n)}, x_5^{(n)}, y^{(n)}) \end{array} \right\}$$

特征值 标签

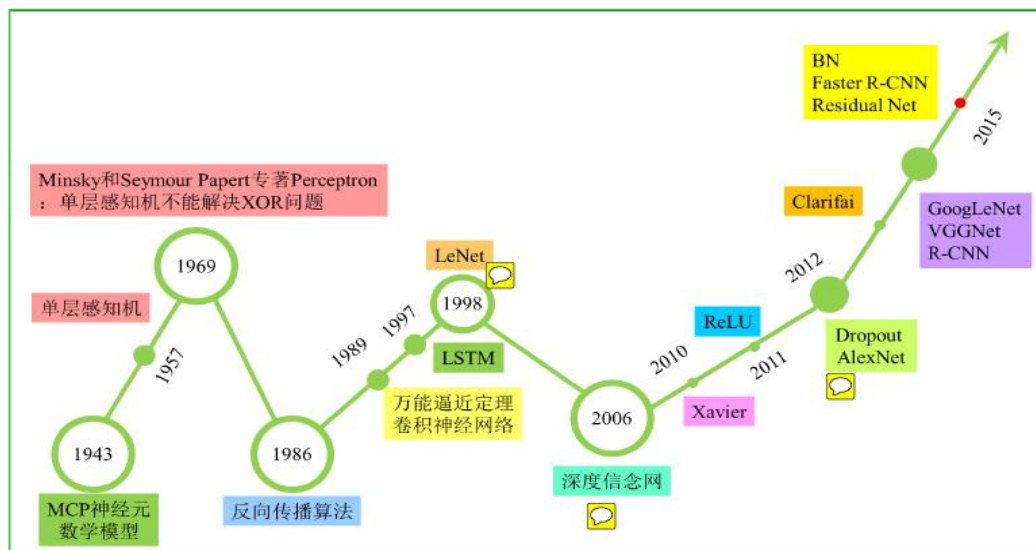
■ 对网络的参数（包含连接权值、偏置）进行学习和调整，以使该网络实现给定样本的输入输出映射关系。

• 第二阶段或称测试阶段：把实际数据输入到网络，网络在误差范围内预测输出结果。

泛化能力：神经网络对未知数据（非训练样本集中的数据）预测的能力；如果不满足**独立同分布假设**，那么训练得到的模型的泛化能力不好。

两个问题 p216:

- 1) 是否存在一个BP神经网络能够逼近给定的样本或者函数。



万能逼近定理表明，含有一个隐藏层的多层感知机可以以任意精度逼近任何的有界连续函数

只要激活函数是非常数、有界、单调递增的连续函数（比如Sigmoid、ReLU），对于任意一个连续函数 $f(x)$ 和任意一个正数 ε ，存在一个具有至少一个隐藏层的神经网络 $g(x)$ 使得对于所有的输入 x ，满足：

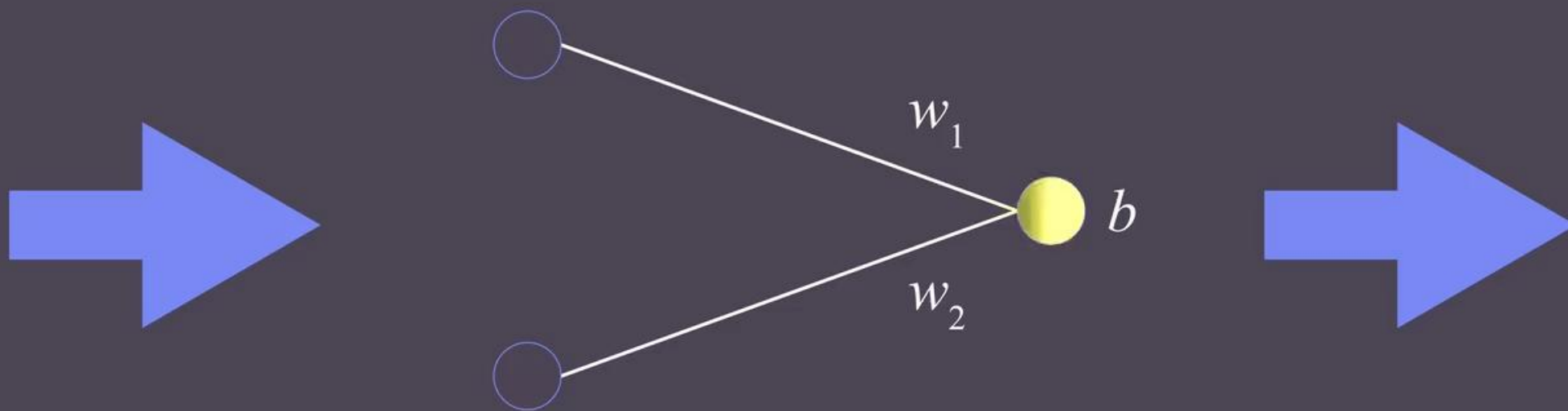
$$|f(x)-g(x)|<\varepsilon$$

换句话说，神经网络可以用来逼近任意连续函数，并且可以达到任意给定的精度要求

- 2) 如何调整BP神经网络的连接权，使网络的输入与输出与给定的样本相同。
- ◆ BP（反向传播）算法最早由Werbos在1974年提出应用到神经网络，D. Rumelhart、Hinton等在1986年发展BP学习算法，实现了Minsky多层感知器的设想。

那就叫小苦吧 bilibili

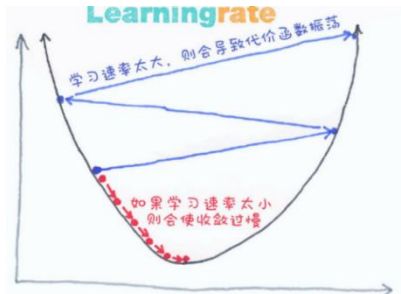
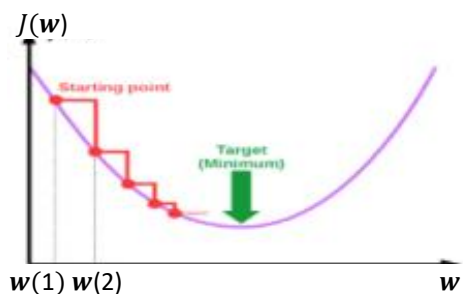
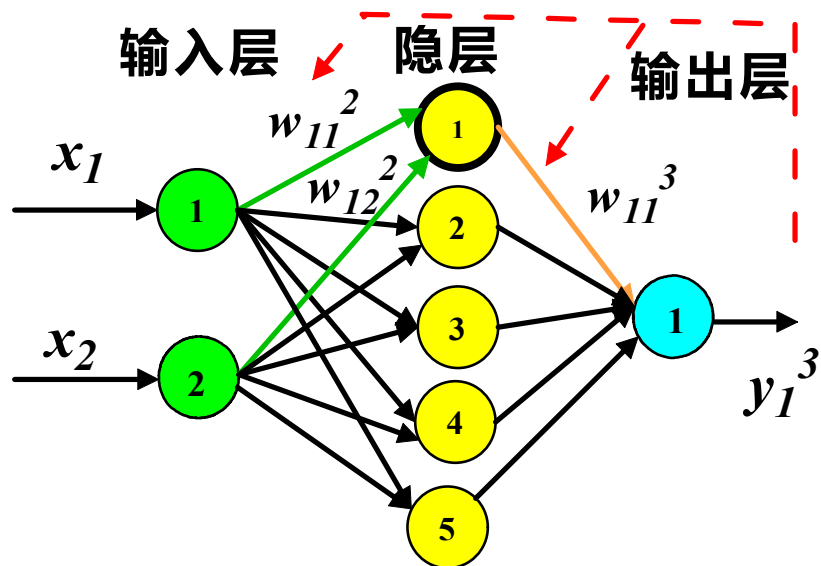
数据正向传递



我们的数据自输入开始向输出传递

3. 训练/学习过程

- 正向传播：输入信息---输入层---隐层---输出层
- 反向传播：误差以某种形式在各层表示，修改各层神经元的权值



目标函数：均方误差损失函数

$$\min J(w, b) = \frac{1}{n} \sum_{i=1}^n L(\hat{y}^{(i)}, y^{(i)}) = \frac{1}{2n} \sum_{i=1}^n (\hat{y}^{(i)} - y^{(i)})^2$$

预测值

真实值

约束条件： $y^k = f^k(W^k y^{k-1} + b^k),$
 $k = 2, 3, \dots, m$

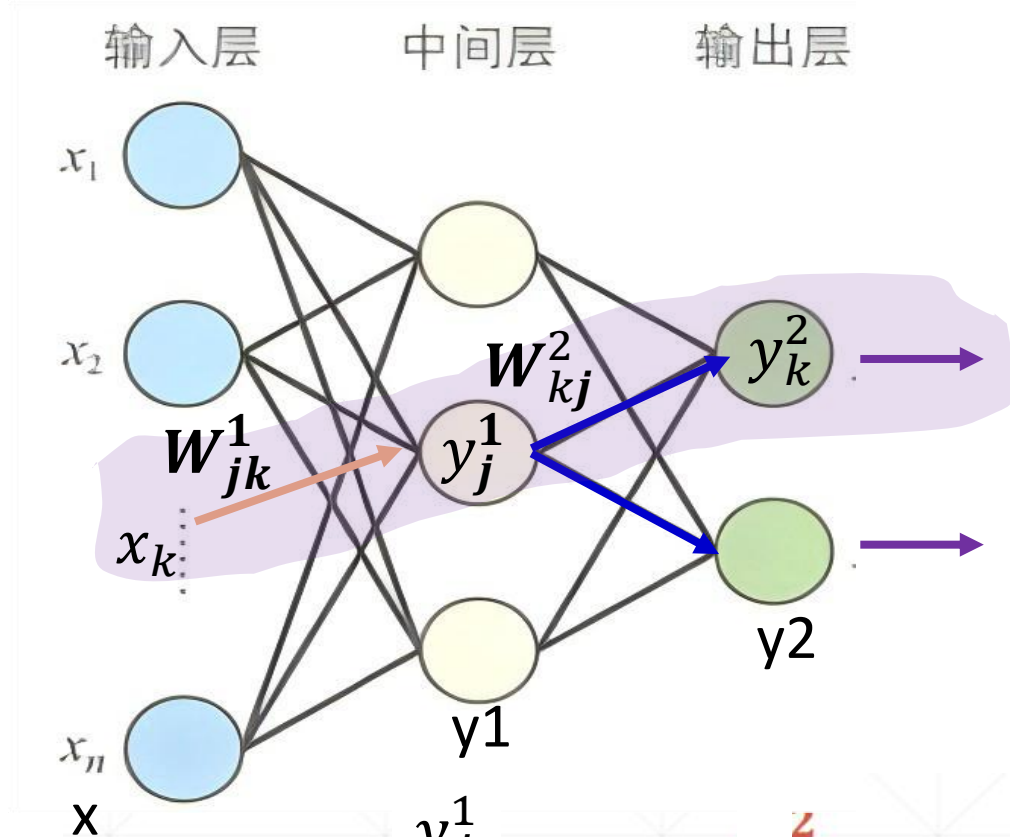
连接权值（参数）的调整：梯度下降法

$$w_{ij}^k(t+1) = w_{ij}^k(t) + \Delta w_{ij}^k(t) = w_{ij}^k(t) - \eta \frac{\partial J(w, b)}{\partial w_{ij}^k(t)}$$

参数
梯度

学习率

学习率越大越好
还是越小越好？



正向传播

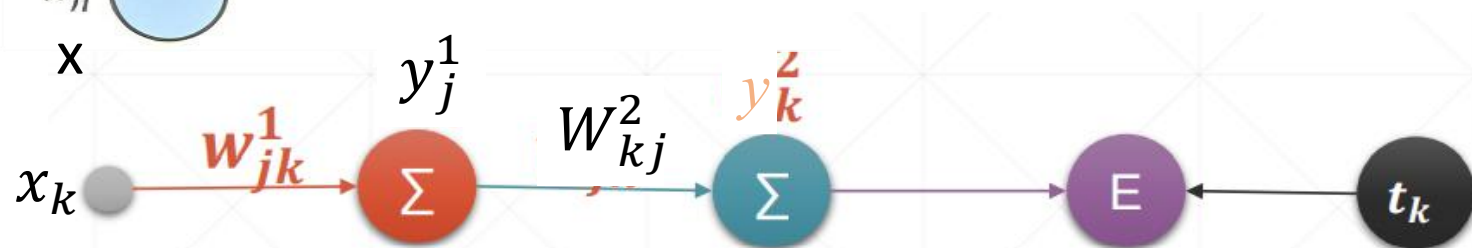
隐藏层输出 $y^1 = f^1(W^1x + b^1)$

输出层输出 $y^2 = f^2(W^2y^1 + b^2)$

目标函数 $J = E = \frac{1}{2} \sum_k e_k^2 = \frac{1}{2} \sum_k (y_k^2 - t_k)^2$

预测值

真实值



梯度消失
梯度爆炸

■ 链式法则（反向传播，计算参数梯度）

$$\frac{\partial E}{\partial w^1_{jk}} = \frac{\partial E}{\partial y^1_j} \frac{\partial y^1_j}{\partial w^1_{jk}} = \frac{\partial E}{\partial y^2_k} \frac{\partial y^2_k}{\partial y^1_j} \frac{\partial y^1_j}{\partial w^1_{jk}} = (y_k^2 - t_k) \times f^2'(\cdot) \times W^2_{kj} \times f^1'(\cdot) \times x_k$$

$$\frac{\partial E}{\partial w_{jk}^1} = \underbrace{(y_k^2 - t_k)}_{\text{误差项}} \cdot \underbrace{f^{2'}(\cdot)}_{\text{输出层激活导数}} \cdot \underbrace{W_{kj}^2}_{\text{后层权重}} \cdot \underbrace{f^{1'}(\cdot)}_{\text{隐藏层激活导数}} \cdot \underbrace{x_k}_{\text{原始输入}}$$

$$\frac{\partial E}{\partial w_{\text{浅层}}} = \text{误差项} \cdot \prod_{\text{所有后续层}} (W_{\text{后层}} \cdot f'(\cdot)) \cdot \text{浅层输入}$$

$$\lambda = W \cdot f'(\cdot)$$

若 $|\lambda| > 1$:

若 $|\lambda| < 1$:

- 假设现有一个MLP（假设无偏置参数，有 m 层）

$$\mathbf{y}^t = \mathbf{f}(\mathbf{W}^t \mathbf{y}^{t-1}) \quad \mathbf{f} \text{ 是一个激活函数}$$

$$\frac{\partial \mathbf{y}^t}{\partial \mathbf{y}^{t-1}} = \text{diag}(\mathbf{f}'(\mathbf{W}^t \mathbf{y}^{t-1})) \mathbf{W}^t \quad \mathbf{f}' \text{ 是 } \mathbf{f} \text{ 的导函数}$$

$\text{diag}(x)$ 把向量 x 的每个元素作为矩阵的对角元素，得到其它元素全为0的对角矩阵

$$\mathbf{W}_{t+1} = \mathbf{W}_t - \eta \nabla_{\mathbf{W}=\mathbf{W}_t} J(\mathbf{W})$$

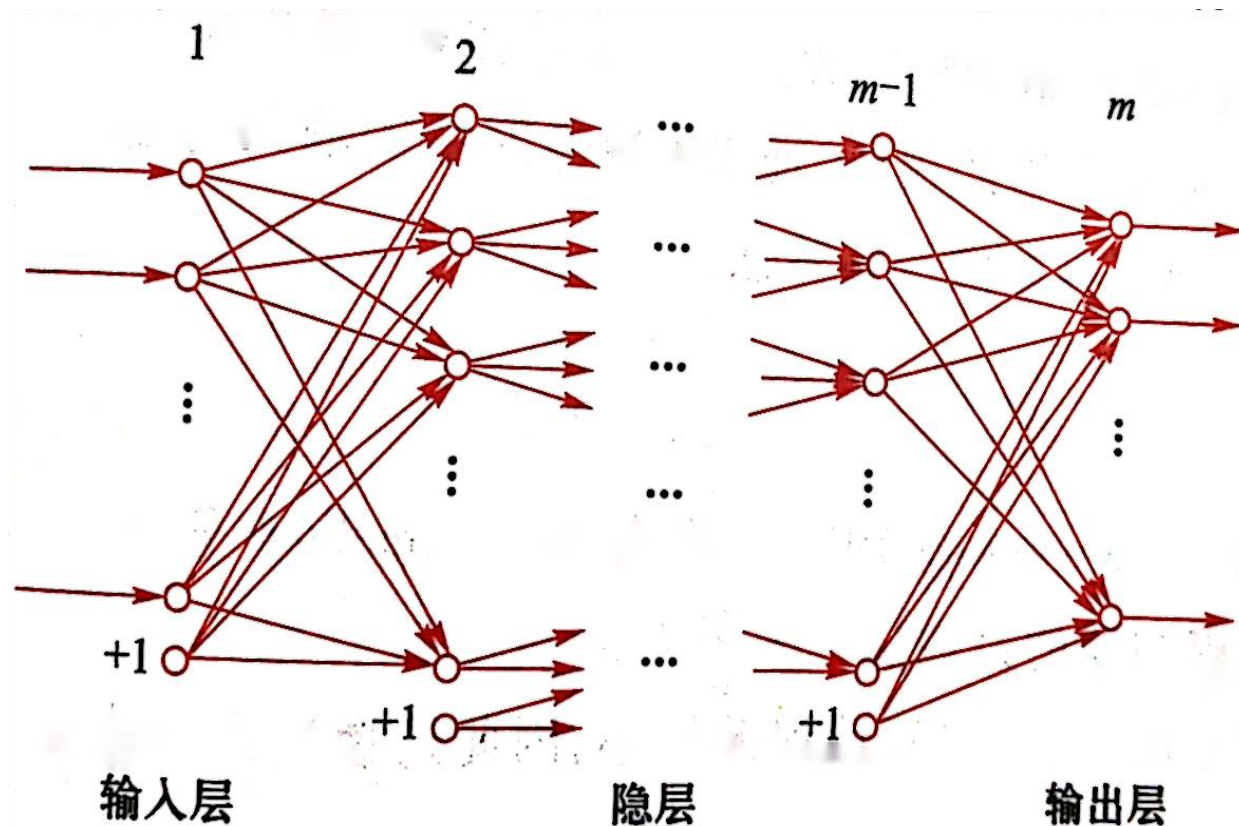
参数梯度

计算损失 J 的梯度 $\mathbf{W}_t$

$$\frac{\partial J}{\partial \mathbf{W}^t} = \frac{\partial J}{\partial \mathbf{y}^m} \underbrace{\frac{\partial \mathbf{y}^m}{\partial \mathbf{y}^{m-1}} \cdots \frac{\partial \mathbf{y}^{t+1}}{\partial \mathbf{y}^t}}_{m-t \text{ 矩阵相乘}} \frac{\partial \mathbf{y}^t}{\partial \mathbf{W}^t}$$

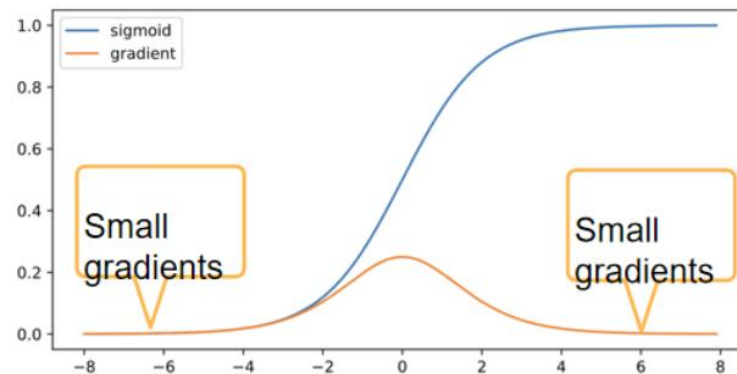
$m-t$ 矩阵相乘

$$\prod_{i=t}^{m-1} \frac{\partial \mathbf{y}^{i+1}}{\partial \mathbf{y}^i} = \prod_{i=t}^{m-1} \text{diag}(\mathbf{f}'(\mathbf{W}^i \mathbf{y}^{i-1})) (\mathbf{W}^i)$$

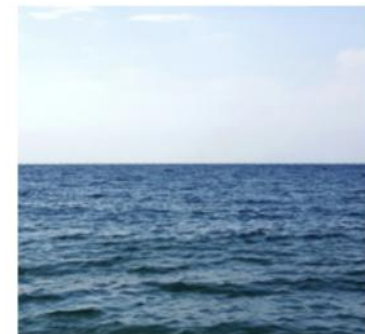


- 激活函数 sigmoid: $f(x) = \frac{1}{1 + e^{-x}} \Rightarrow f'(x) = f(x)(1 - f(x))$

$$\prod_{i=t}^{m-1} \frac{\partial y^{i+1}}{\partial y^i} = \prod_{i=t}^{m-1} \text{diag}(f'(\mathbf{W}^i y^{i-1})) (\mathbf{W}^i)^T$$



梯度消失



$$0.25^8 \approx 2.3 \times 10^{-10}$$

- 激活函数 ReLU:

$$f(x) = \max(0, x) \Rightarrow f'(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{otherwise} \end{cases}$$

$$\prod_{i=t}^{m-1} (\mathbf{W}^i)^T \text{ 为来自 } \prod_{i=t}^{m-1} \frac{\partial y^{i+1}}{\partial y^i} = \prod_{i=t}^{m-1} \text{diag}(f'(\mathbf{W}^i y^{i-1})) (\mathbf{W}^i)^T$$

思考: W 初始化值过大或过小会如何?

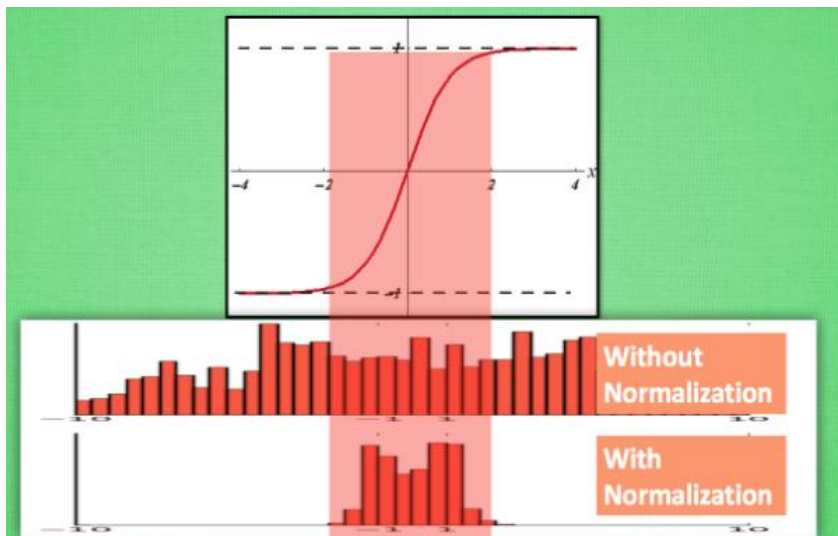
梯度爆炸



$$1.5^{100} \approx 4 \times 10^{17}$$

Batch Normalization (Ioffe et al., 2015)

$$\mathbf{y}^t = f(\mathbf{W}^t \mathbf{y}^{t-1})$$



Batch Normalization: 对于每个隐层神经元，把激活函数非线性映射后取值区间向极限饱和区靠拢的输入分布强制拉回到均值为0、方差为1的标准的正态分布，使得激活函数的输入值落入对输入比较敏感的区域，以避免梯度消失，加快训练速度。

```
nn.BatchNorm2d(self, num_features, eps=1e-5, momentum=0.1,
affine=True, track_running_stats=True)
```

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_1 \dots x_m\}$;

Parameters to be learned: γ, β

Output: $\{y_i = \text{BN}_{\gamma, \beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad // \text{ mini-batch mean}$$

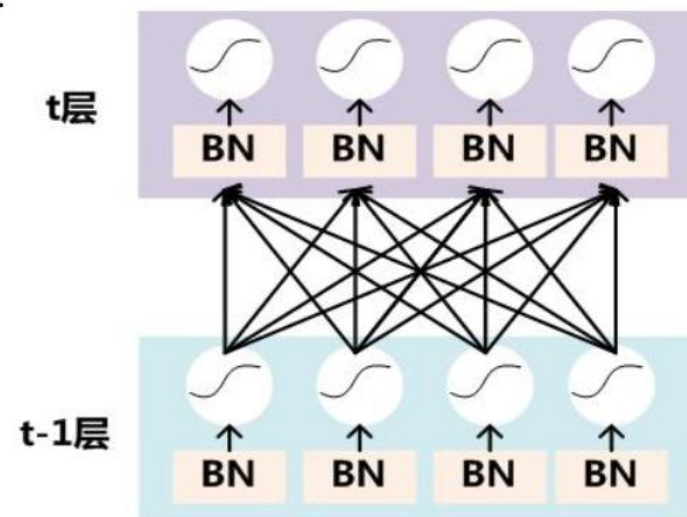
$$\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2 \quad // \text{ mini-batch variance}$$

$$\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \quad // \text{ normalize}$$

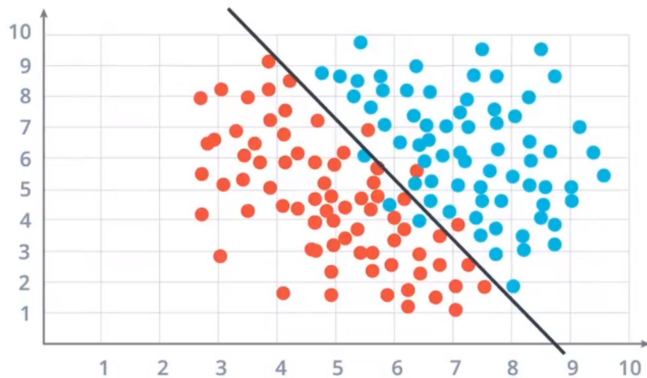
$$y_i \leftarrow \gamma \hat{x}_i + \beta \equiv \text{BN}_{\gamma, \beta}(x_i) \quad // \text{ scale and shift}$$

Algorithm 1: Batch Normalizing Transform, applied to activation x over a mini-batch.

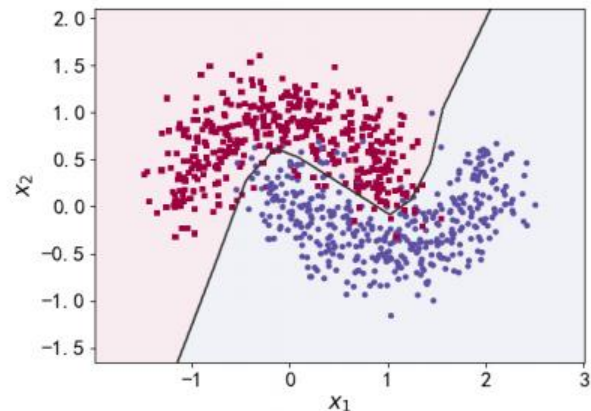
scale和shift的参数通过训练学习到的，用来对变换后的激活反变换，增强网络非线性表达能力



感知器的学习过程与求判决函数的过程是等价的



单层感知器只能用于解决线性可分的问题



多层感知器网络可实现任何模式分类

多层网络的权值如何确定在感知器上没有得到解决?

定义模型

```
model = nn.Sequential(
    nn.Linear(2, 32),      # 输入层2个神经元，隐藏层32个神经元
    nn.ReLU(),            # ReLU激活函数
    nn.Linear(32, 1),      # 输出层1个神经元
    nn.Sigmoid()          # Sigmoid激活函数用于二分类
)
```

定义损失函数和优化器（相当于Keras的compile）

```
criterion = nn.BCELoss() # 二元交叉熵损失
optimizer = optim.Adam(model.parameters()) # Adam优化器
```



Deep Learning is so simple

- 正向传播
- 反向传播
- 梯度下降

全连接神经网络模型: $y^k = f^k(W^k y^{k-1} + b^k)$, $k = 2, \dots, m$

1. 给出训练样本集 $\{x^{(i)}, y^{(i)}\}$ 、最大训练周期（迭代步数） $epoch$ 、学习率 lr 、批量大小 p 等
2. 根据输入一批样本 $\{x^{(i)}\}$ 以及NN的模型结构，**前向传播**(forward propagation)
3. 由前向传播结果**计算模型输出** $\{\hat{y}^{(i)}\}$ ，并**计算代价函数** $J(w) = \frac{1}{p} \sum_{i=1}^p L(\hat{y}^{(i)}, y^{(i)})$
4. 根据代价函数进行**反向传播**(back propagation)，**计算所有参数梯度**
5. 根据参数梯度进行**梯度下降**，**调整模型参数**，返回步骤2

$$W_{t+1} = W_t - \eta \nabla_{W=W_t} J(W)$$

参数
梯度

$$\frac{\partial J}{\partial W^k} = \frac{\partial J}{\partial y^m} \frac{\partial y^m}{\partial y^{m-1}} \cdots \frac{\partial y^{k+1}}{\partial y^k} \frac{\partial y^k}{\partial W^k}$$

$\prod_{i=k}^{m-1} \text{diag}(f'(W^i y^{i-1}))(W^i)$

例如用BP神经网络（全连接神经网络）预测房价

```
## 输出我们的网络结构
mlp1 = MLPmodel()
print(mlp1)

MLPmodel(
  (hidden1): Linear(in_features=13, out_features=10, bias=True)
  (active1): ReLU()
  (hidden2): Linear(in_features=10, out_features=10, bias=True)
  (active2): ReLU()
  (regression): Linear(in_features=10, out_features=1, bias=True)
)

## 对回归模型mlp1进行训练并输出损失函数的变化情况
# 定义优化器和损失函数
optimizer = SGD(mlp1.parameters(), lr=0.001)
loss_func = nn.MSELoss() # 最小平方根误差
train_loss_all = [] ## 输出每个批次训练的损失函数
## 进行训练，并输出每次迭代的损失函数
for epoch in range(30):
    ## 对训练数据的迭代器进行迭代计算
    for step, (b_x, b_y) in enumerate(train_loader):
        output = mlp1(b_x).flatten() # MLP在训练batch上的输出
        train_loss = loss_func(output, b_y) # 均方误差损失
        optimizer.zero_grad() # 每个迭代步的梯度初始化为0
        train_loss.backward() # 损失的后向传播，计算梯度
        optimizer.step() # 使用梯度进行优化
        train_loss_all.append(train_loss.item())
```

使用继承Module

```
## 使用继承Module的形式定义全连接神经网络
class MLPmodel(nn.Module):
    def __init__(self):
        super(MLPmodel, self).__init__()
        ## 定义第1个隐藏层
        self.hidden1 = nn.Linear(
            in_features = 13, ## 第一个隐藏层的输入，数据的特征数
            out_features = 10, ## 第一个隐藏层的输出，神经元的数量
            bias=True, ## 默认会有偏置
        )
        self.active1 = nn.ReLU()
        ## 定义第2个隐藏层
        self.hidden2 = nn.Linear(10, 10)
        self.active2 = nn.ReLU()
        ## 定义预测回归层
        self.regression = nn.Linear(10, 1)

    ## 定义网络的向前传播路径
    def forward(self, x):
        x = self.hidden1(x)
        x = self.active1(x)
        x = self.hidden2(x)
        x = self.active2(x)
        output = self.regression(x)
        ## 输出为output
        return output
```



训练过程三部曲

- 正向传播
- 反向传播
- 梯度下降



使用继承Module

使用继承Module的形式定义全连接神经网络

```
class MLPmodel(nn.Module):
    def __init__(self):
        super(MLPmodel, self).__init__()
        ## 定义第一个隐藏层
        self.hidden1 = nn.Linear(
            in_features = 13, ## 第一个隐藏层的输入, 数据的特征数
            out_features = 10, ## 第一个隐藏层的输出, 神经元的数量
            bias=True, ## 默认会有偏置
        )
        self.active1 = nn.ReLU()
        ## 定义第二个隐藏层
        self.hidden2 = nn.Linear(10, 10)
        self.active2 = nn.ReLU()
        ## 定义预测回归层
        self.regression = nn.Linear(10, 1)

    ## 定义网络的向前传播路径
    def forward(self, x):
        x = self.hidden1(x)
        x = self.active1(x)
        x = self.hidden2(x)
        x = self.active2(x)
        output = self.regression(x)
        ## 输出为output
        return output
```

输出我们的网络结构

```
mlp1 = MLPmodel()
print(mlp1)

MLPmodel(
  (hidden1): Linear(in_features=13, out_features=10, bias=True)
  (active1): ReLU()
  (hidden2): Linear(in_features=10, out_features=10, bias=True)
  (active2): ReLU()
  (regression): Linear(in_features=10, out_features=1, bias=True)
)
```

使用nn.Sequential

使用定义网络时使用nn.Sequential的形式

```
class MLPmodel2(nn.Module):
    def __init__(self):
        super(MLPmodel2, self).__init__()
        ## 定义隐藏层
        self.hidden = nn.Sequential(
            nn.Linear(13, 10),
            nn.ReLU(),
            nn.Linear(10, 10),
            nn.ReLU(),
        )
        ## 预测回归层
        self.regression = nn.Linear(10, 1)

    ## 定义网络的向前传播路径
    def forward(self, x):
        x = self.hidden(x)
        output = self.regression(x)
        return output
```

输出我们的网络结构

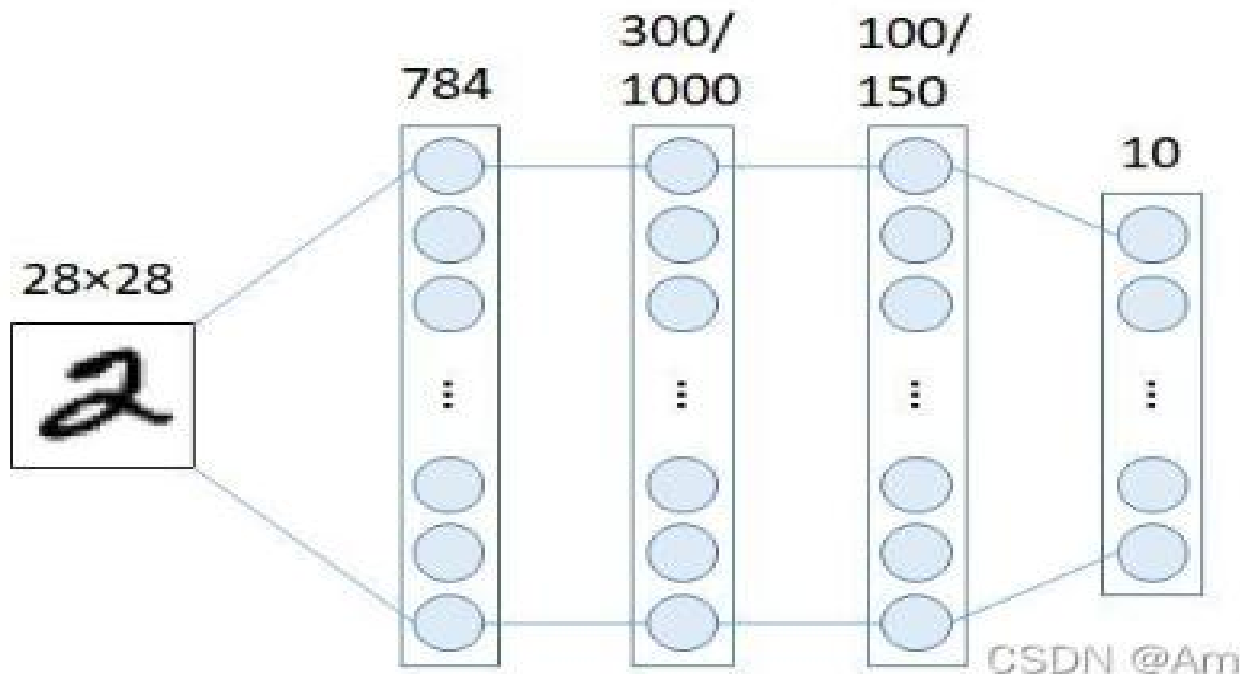
```
mlp2 = MLPmodel2()
print(mlp2)

MLPmodel2(
  (hidden): Sequential(
    (0): Linear(in_features=13, out_features=10, bias=True)
    (1): ReLU()
    (2): Linear(in_features=10, out_features=10, bias=True)
    (3): ReLU()
  )
  (regression): Linear(in_features=10, out_features=1, bias=True)
)
```

手写数字识别采用含有两个隐层的全连接神经网络



MNIST数据集
(Yann LeCun,
6w + 1w)



- 输出层采用**softmax**函数
- 采用交叉熵损失函数
- 小批量梯度下降法

28×28 - 300 - 100-10 网络，测试数据的错误率为3.05%

28×28 - 1000 - 150 - 10 网络，测试数据错误率为2.95%

• Softmax函数

• 特点

- 当有多个输入的时候通过概率求得那个输入能够胜出

• 用法

- 用于多分类神经网络输出
- 用于多分类问题的最后一层

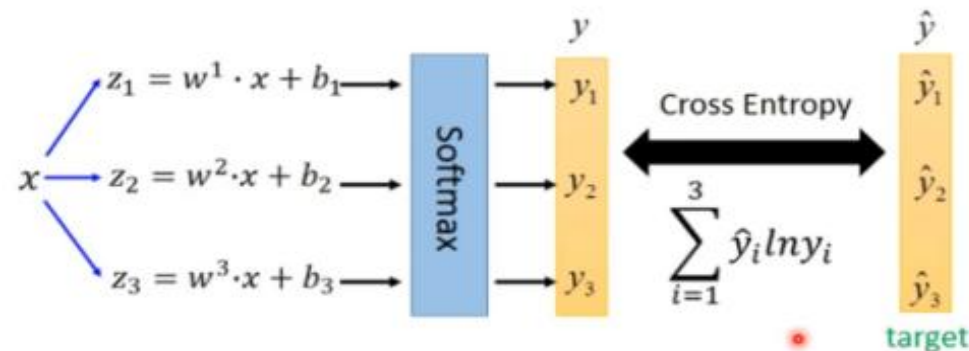
• 缺点

- 只用于多分类问题

如果将分类问题中“一个样例属于某一个类别”看成一个概率事件，那么训练数据的正确答案就是一个概率分布

全连接神经网络模型:

$$y^k = f^k(W^k y^{k-1} + b^k), \quad k = 2, \dots, m$$



If $x \in \text{class 1}$

If $x \in \text{class 2}$

If $x \in \text{class 3}$

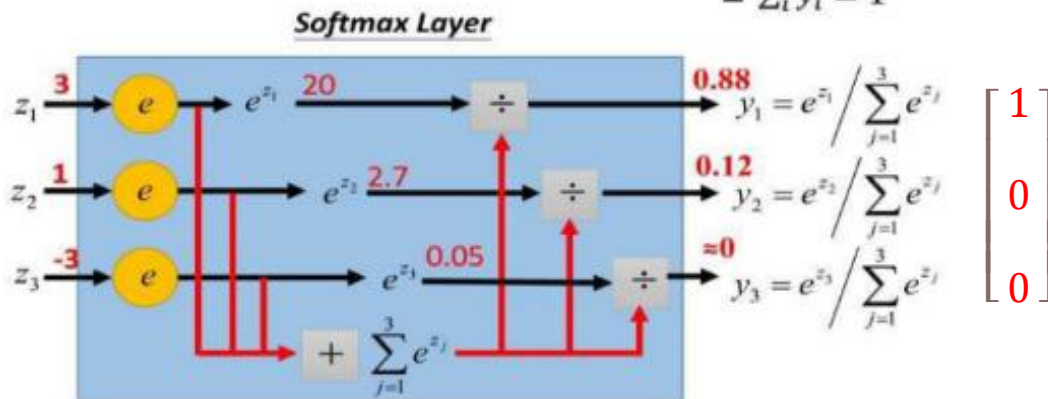
onehot
编码

$$\hat{y} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$$

$$\hat{y} = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$$

$$\hat{y} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

• Softmax layer as the output layer



Probability:

■ $1 > y_i > 0$

■ $\sum_i y_i = 1$

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1} e^{z_j}}$$

Softmax层: 将神经网络的输出变成一个概率分布

Softmax层：将神经网络的输出变成一个概率分布

- 分类问题常用的损失函数：**交叉熵**(两个概率分布之间的距离)

$$H(p, q) = - \sum_x p(x) \log q(x)$$

p : 正确答案
 q : 预测输出

- 某个样例的**正确答案** $(0, 0, 1)$ ，经Softmax层后的两个**预测输出**为 $(0.3, 0.3, 0.4)$ 和 $(0.1, 0.2, 0.7)$ ，那么交叉熵为

$$H((0, 0, 1), (0.3, 0.3, 0.4)) = -(0 * \log 0.3 + 0 * \log 0.3 + 1 * \log 0.4) \approx 0.4$$

$$H((0, 0, 1), (0.1, 0.2, 0.7)) = -(0 * \log 0.1 + 0 * \log 0.2 + 1 * \log 0.7) \approx 0.2$$

Softmax层的交叉
熵代价函数：

$$J(w, b) = - \frac{1}{n} \sum_{i=1}^n \hat{y}^{(i)} \log y^{(i)}$$

n : 样本数

真实值

预测值：

$$y^{(i)} = \text{soft max}(z_i) = \frac{e^{z_i}}{\sum_{j=1} e^{z_j}}$$

```
Loss = nn.CrossEntropyLoss(y_pred, y_true)
```

`nn.CrossEntropyLoss()`其内部会自动加上Softmax层,将softmax-log-NLLLoss合并

根据万能逼近定理，以下哪项正确？

- ☐ A 只有一个隐藏层的神经网络不能逼近任意连续函数
- ☐ B 含有一个隐藏层的神经网络可以任意精度逼近任何有界连续函数
- ☐ C 多层感知器无法解决非线性问题
- ☐ D 多层感知器不适用于二分类问题

提交

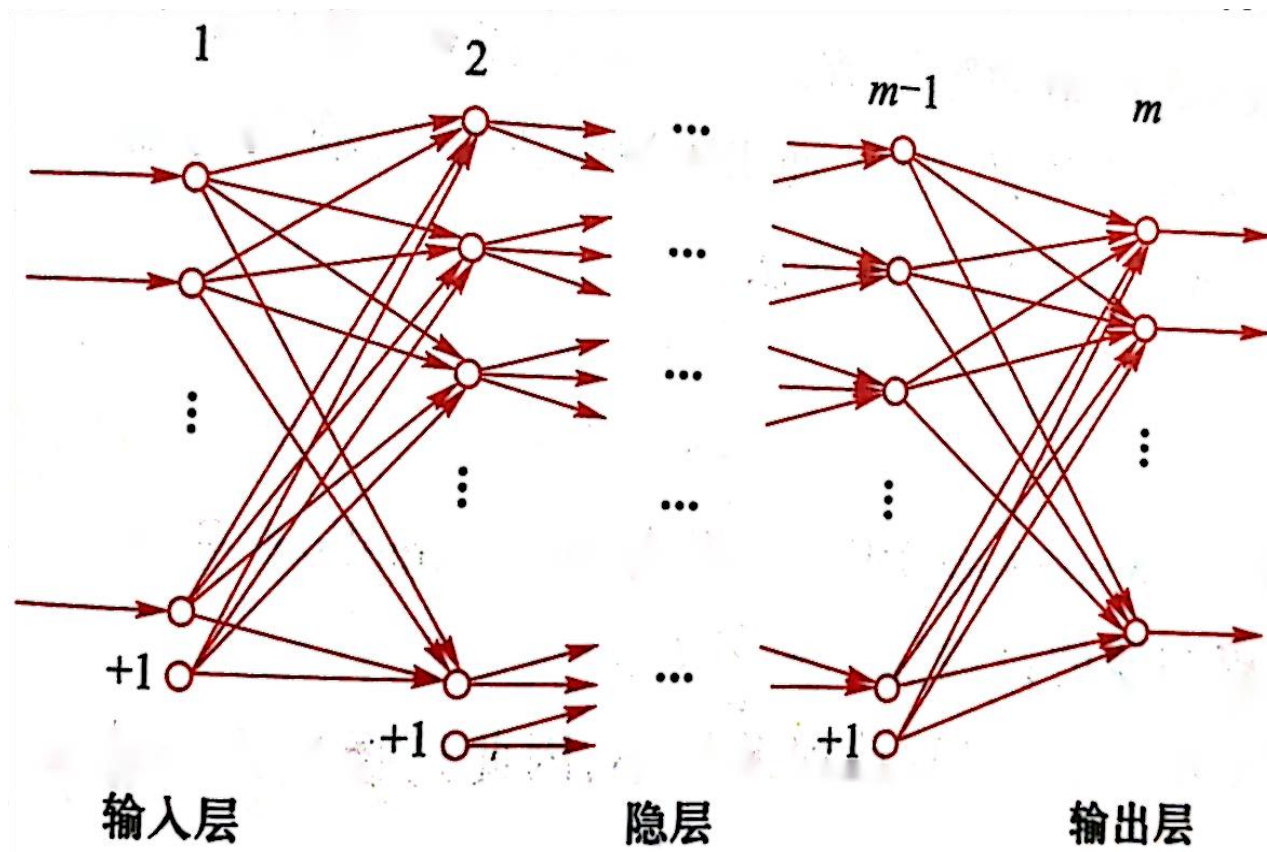
根据万能逼近定理，以下哪项正确？

- ☐ A 只有一个隐藏层的神经网络不能逼近任意连续函数
- ☒ B 含有一个隐藏层的神经网络可以任意精度逼近任何有界连续函数
- ☐ C 多层感知器无法解决非线性问题
- ☐ D 多层感知器不适用于二分类问题

提交

(1) 隐层数、隐层神经元数、激活函数、学习率等确定：目前尚无理论指导。

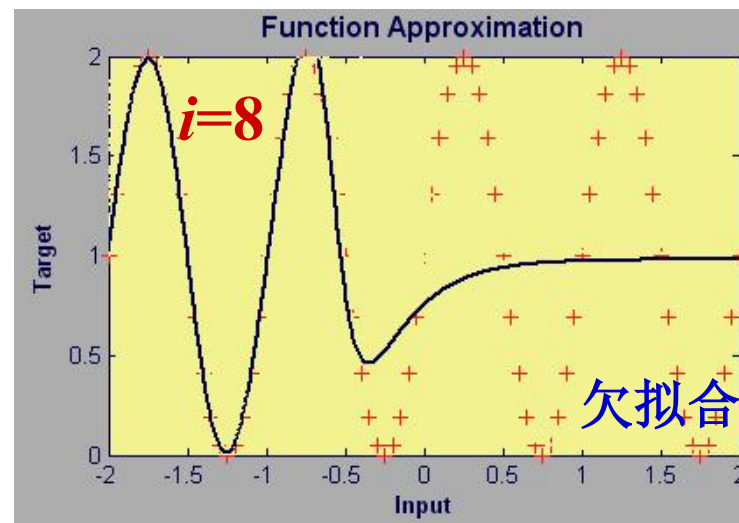
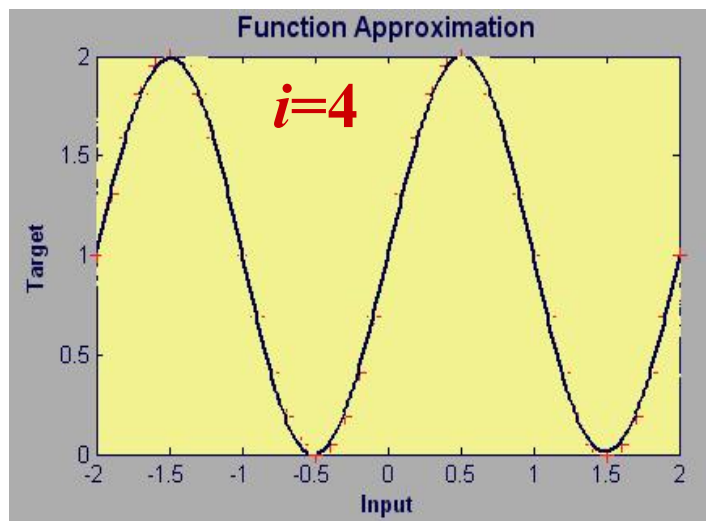
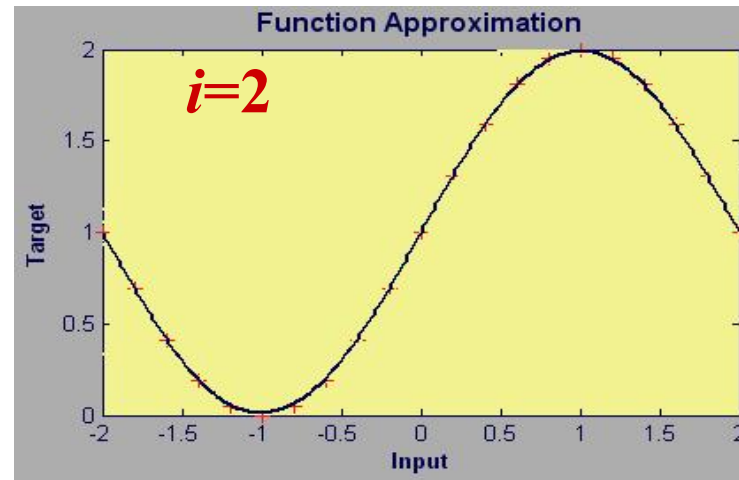
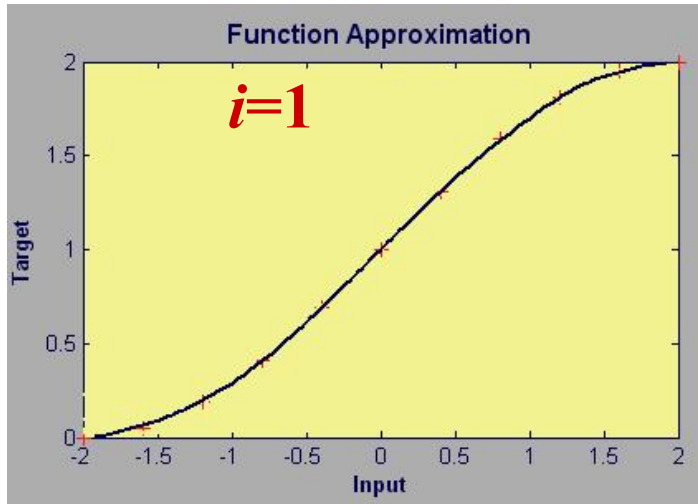
万能逼近定理： 含有一个隐藏层的多层感知机可以以任意精度逼近任何的有界连续函数



• BP神经网络结构的选择

网络要逼近的函数： $y = 1 + \sin(\frac{i\pi}{4}x)$, $-2 \leq x \leq 2$

BP网络：1-3-1，隐层为Sigmoid型函数，输出层为线性函数

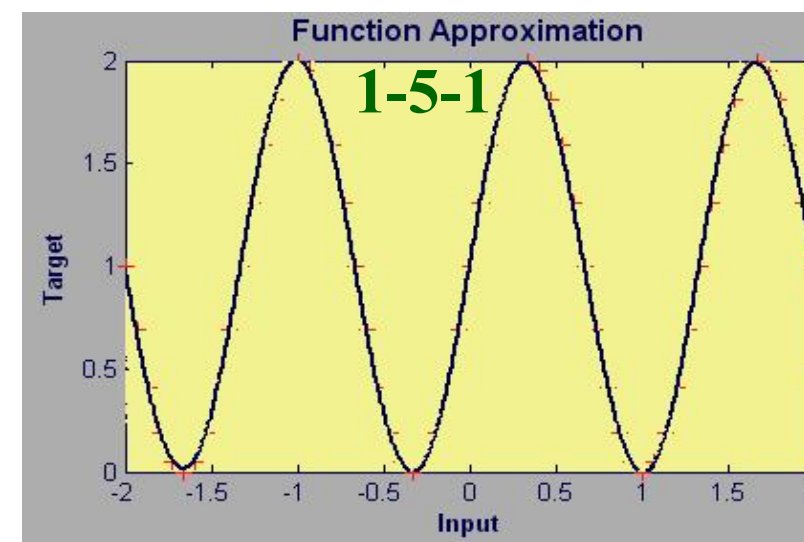
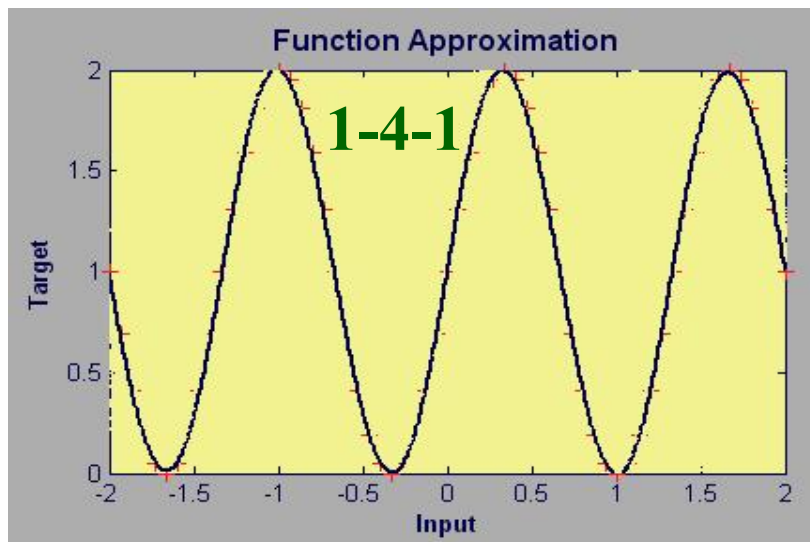
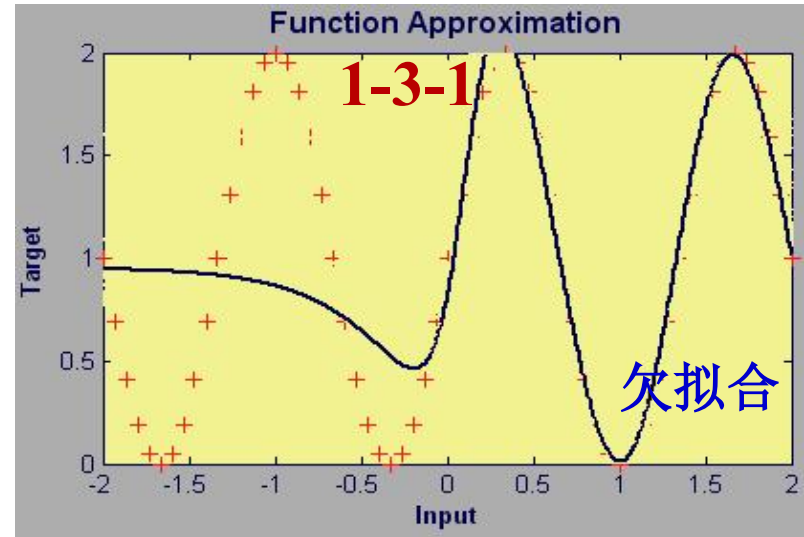
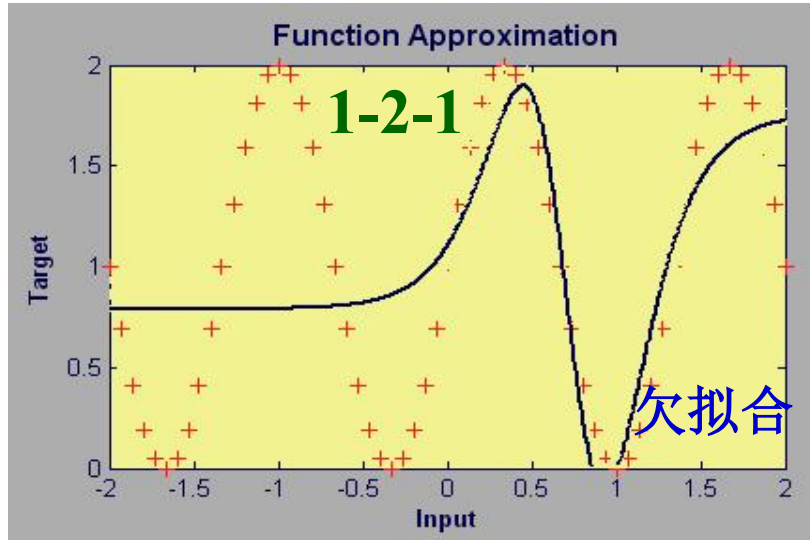


回归问题越复杂，
隐层的神经元数
就需要越多

• BP神经网络结构的选择

网络要逼近的函数： $y = 1 + \sin(\frac{6\pi}{4}x)$, $-2 \leq x \leq 2$

BP网络：1-?-1，隐层为S型函数，输出层为线性函数

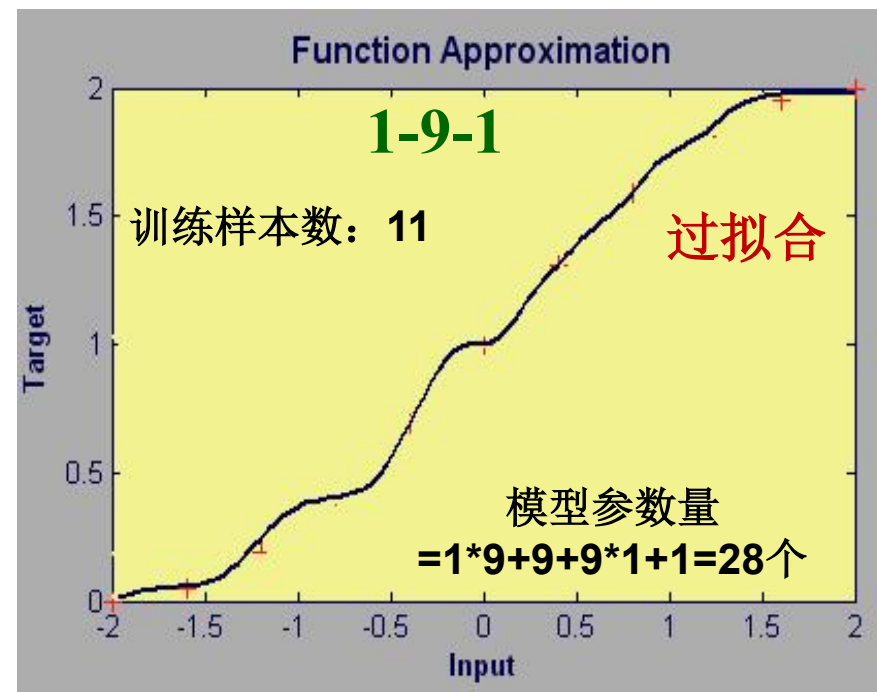
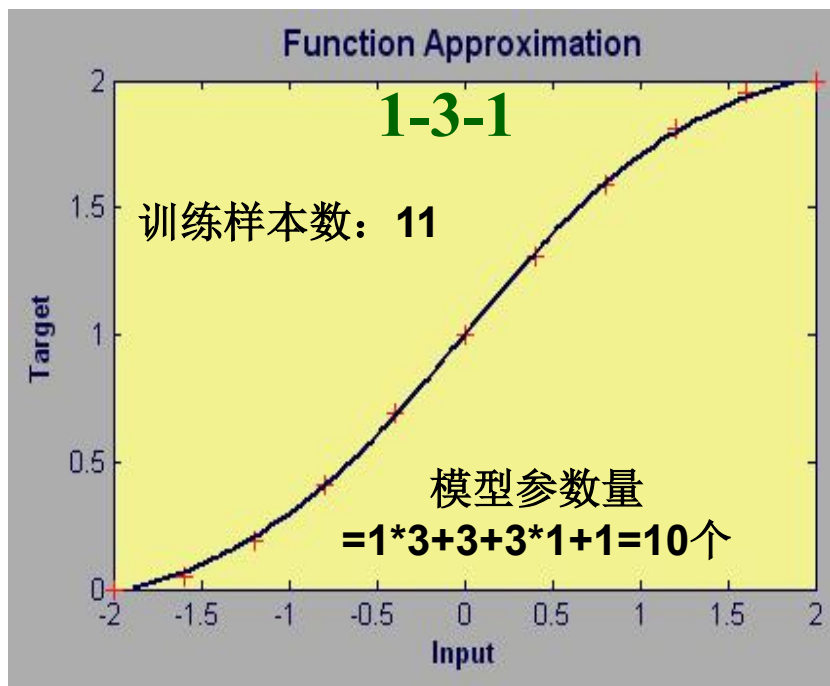


只要一个更小的网络能工作，就不要使用更大的网络（Ockham的“剃刀”）。

- 推广（泛化能力）

网络要逼近的函数： $y = 1 + \sin(\frac{\pi}{4}x)$, $-2 \leq x \leq 2$

BP网络：1-?-1，隐层为对数S型函数，输出层为线性函数



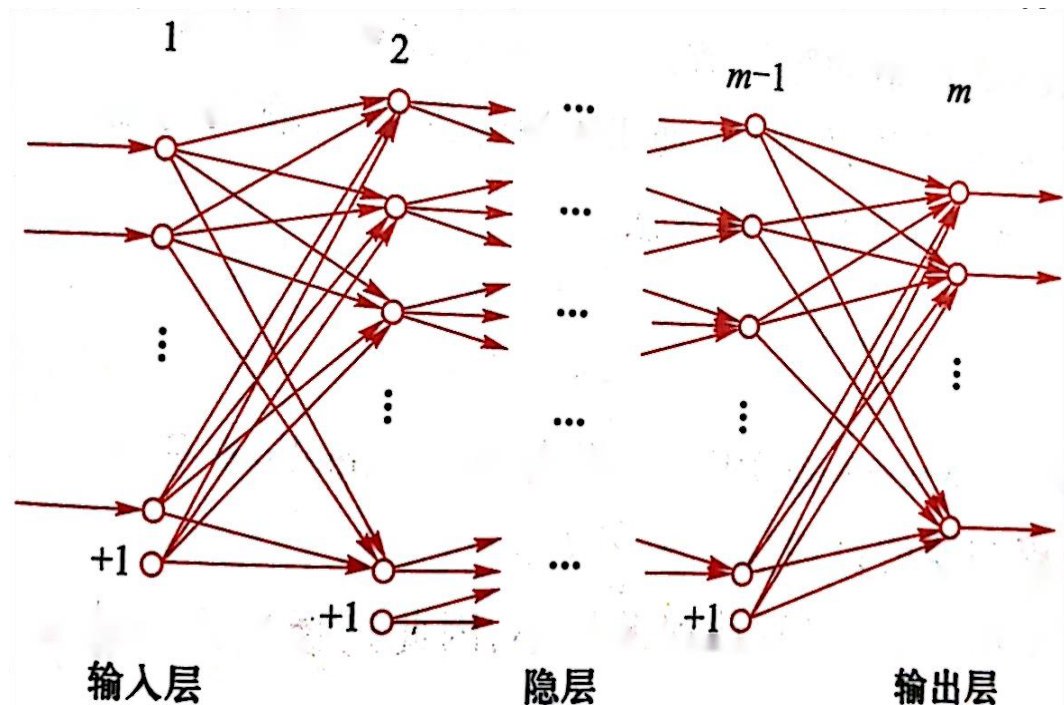
一个BP神经网络（浅层）要能被推广，一般应具有比训练集样本数少的参数量。

(1) 隐层数、隐层神经元数、激活函数、学习率等确定：目前尚无理论指导。

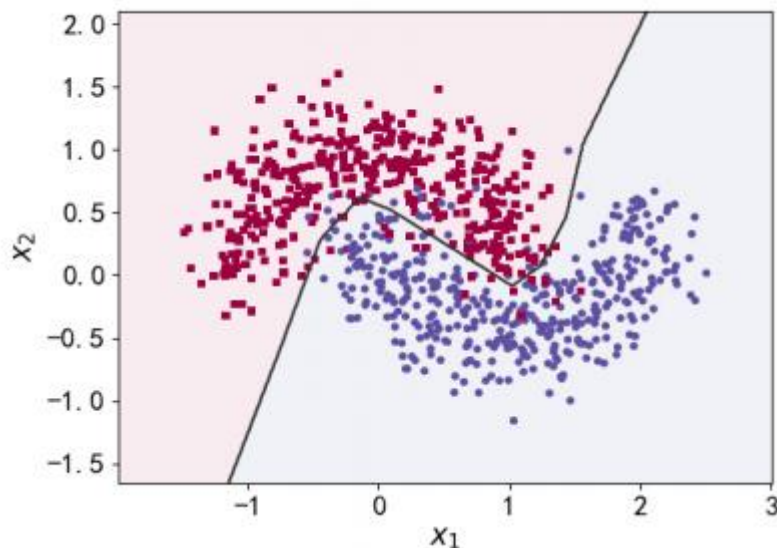
- 网络结构的选择

多层BP神经网络可用来逼近几乎任一个函数，只要在隐层中有足够的神经元。
但通常并不能说，多少层或多少神经元就足以得到足够的性能。

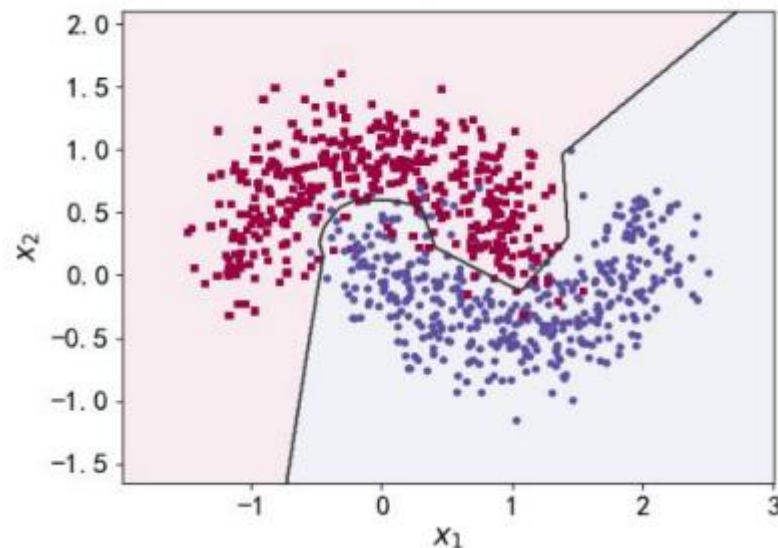
p216万能逼近定理： 含有一个隐藏层的多层感知机可以以任意精度逼近任何的有界连续函数



全连接多层神经网络: $y^k = f(W^k y^{k-1} + b^k)$, $k = 2, \dots, m$ 层数: 不包括输入层

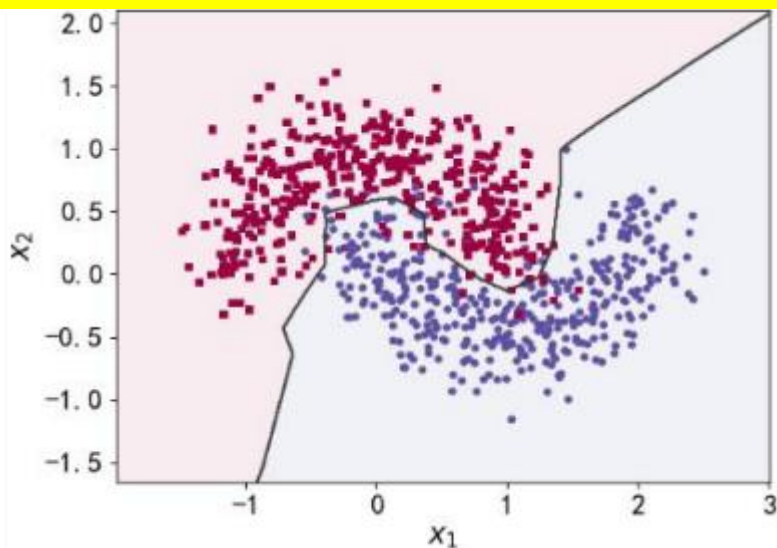


(a) 网络层数: 2

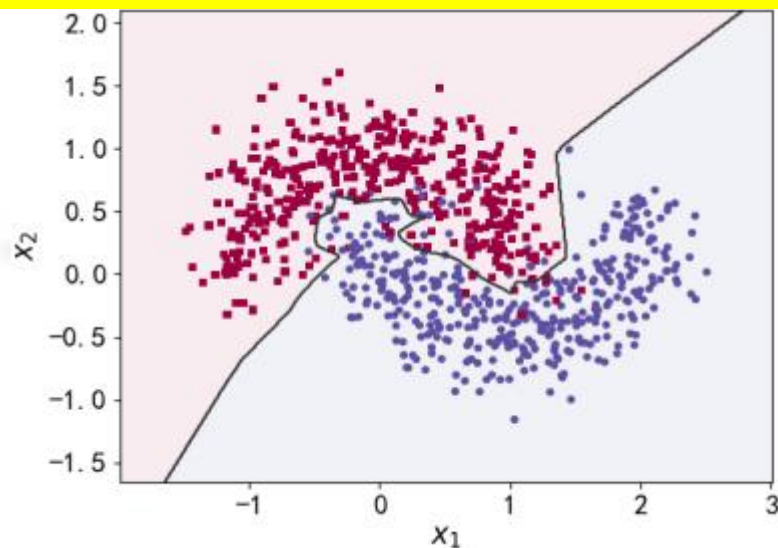


(b) 网络层数: 3

随着层数增加, 模型复杂度增加了, 模型的表达能力增加了, 但易过拟合



(c) 网络层数: 4

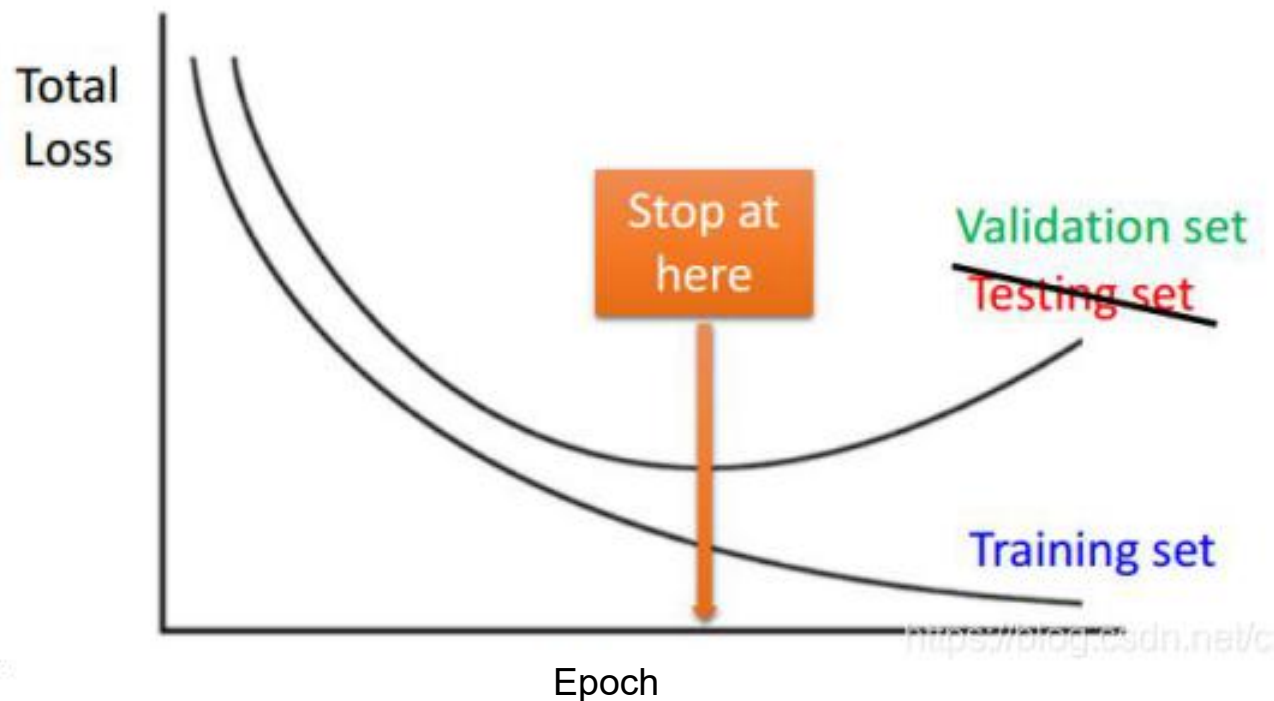


(d) 网络层数: 6

过拟合的原因：在训练数据不够多时，或当一个模型很复杂时，或者训练数据有噪声，**overtraining**时，即模型可以很好地“记忆”每一个训练数据中的随机噪声的部分，而忘记了要去“学习”训练数据中通用的趋势。

✓ 解决方法1：提前终止训练

- ◆ 当代价函数在验证集上不再减小时，训练就该停止了。
- ◆ 不能一味追求训练集的误差减小。

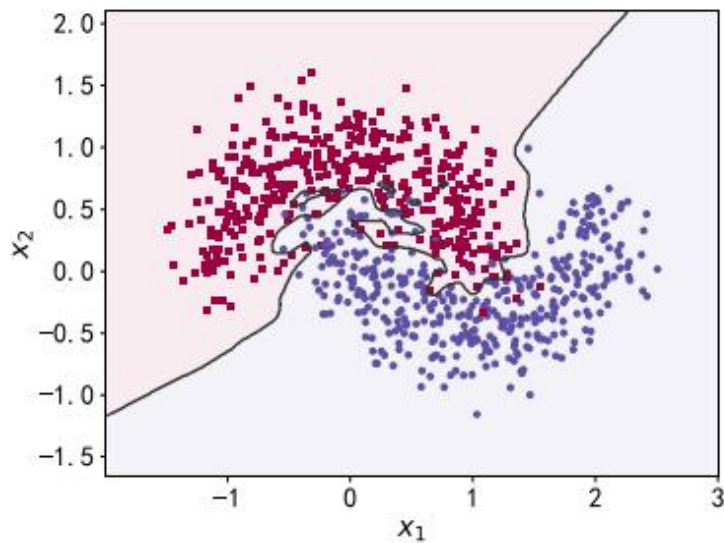


✓ 解决方法2: 正则化 (Regularization)

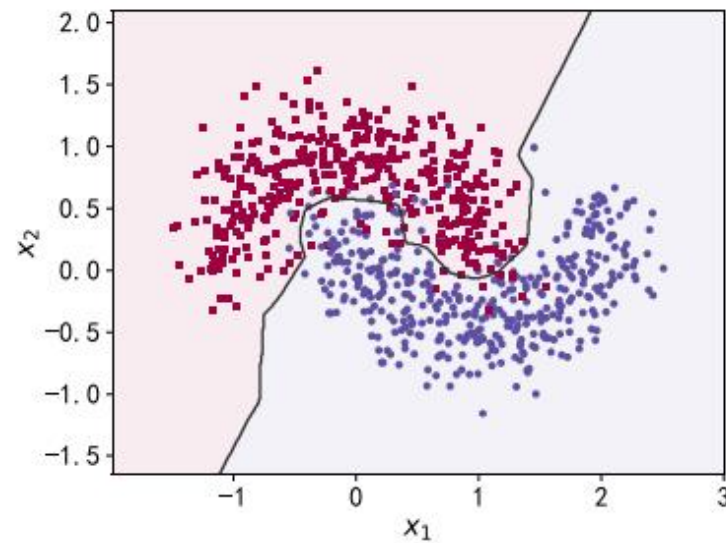
目标函数 = 代价函数 + 正则化项 (如: $\lambda \sum W^2$)

在目标函数中加入刻画模型复杂程度的指标, 基本思想: 通过限制权重的大小, 使得模型不能任意拟合训练数据中的随机噪声, 例如L2正则化:

正则化系数 λ : 模型复杂程度在目标函数中的比例

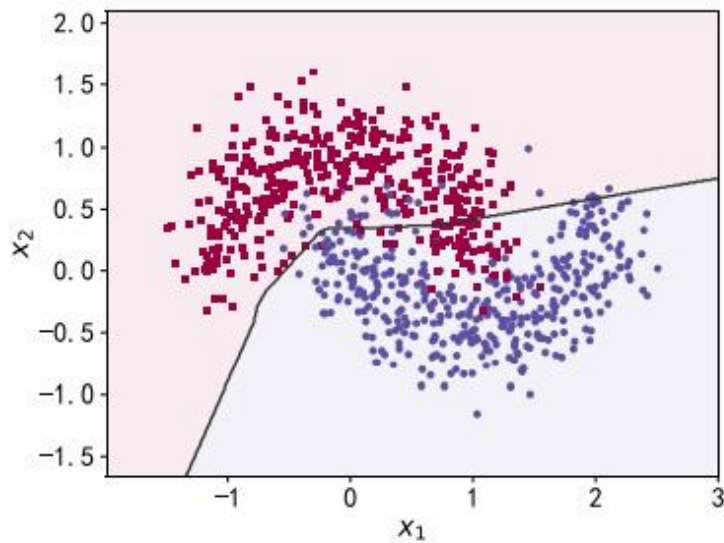


正则化系数: 0.00001

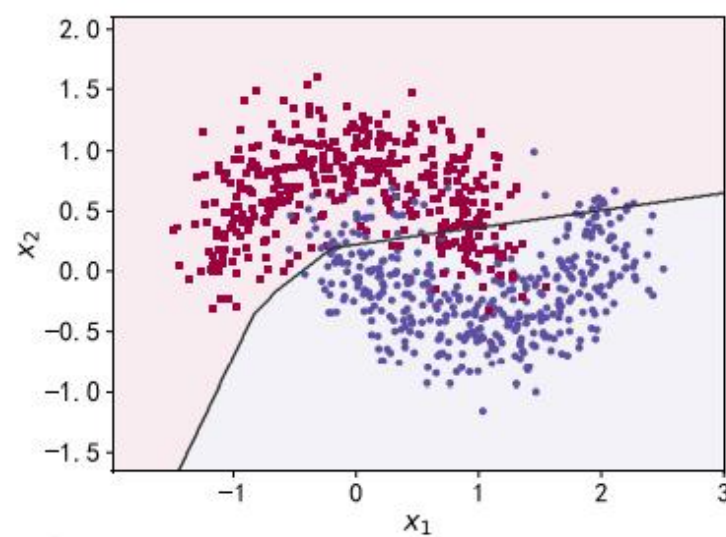


正则化系数: 0.001

`optimizer = optim.Adam(model.parameters, lr=learning_rate, weight_decay=0.1)`



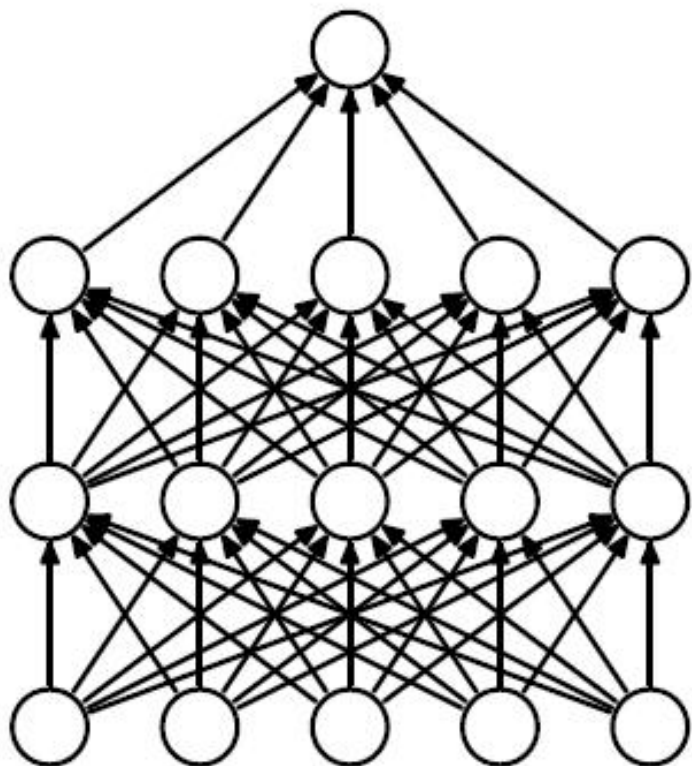
正则化系数: 0.1



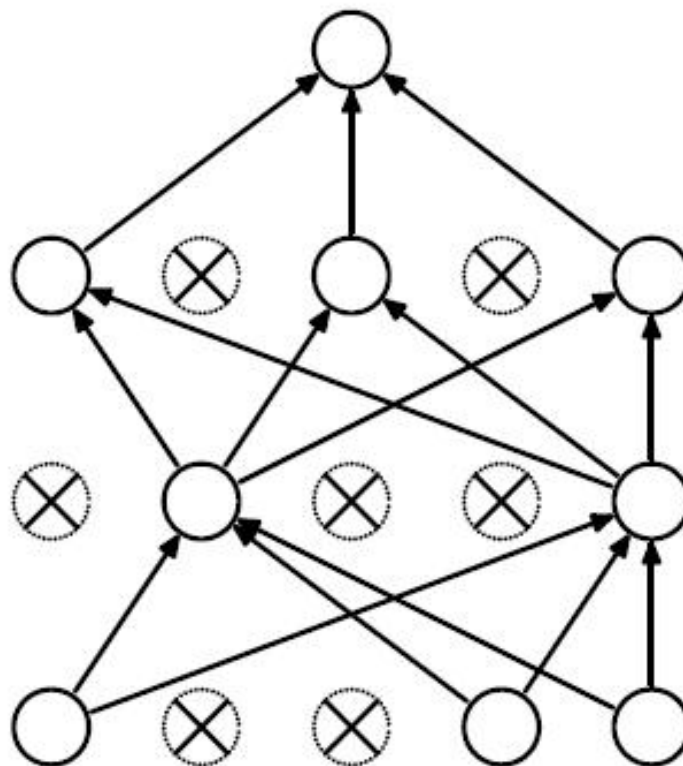
正则化系数: 0.13

✓ 解决方法3: **Dropout**方法

让机器少学点，比如使用Dropout()方法去除一定比例的学习结果。



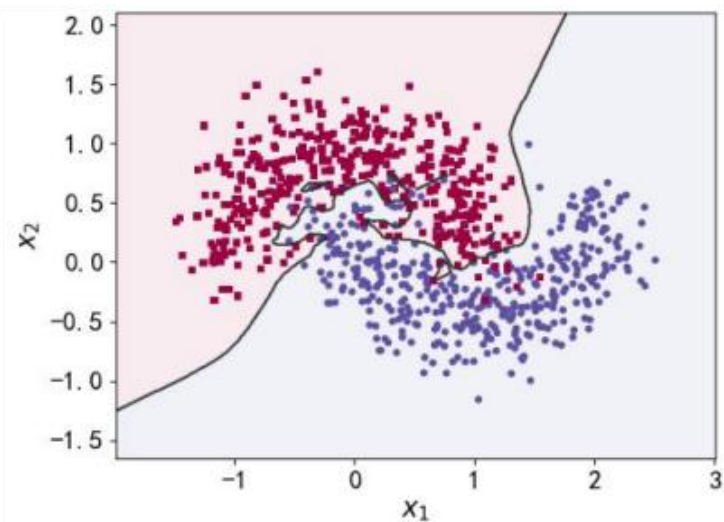
(a) 全连接神经网络



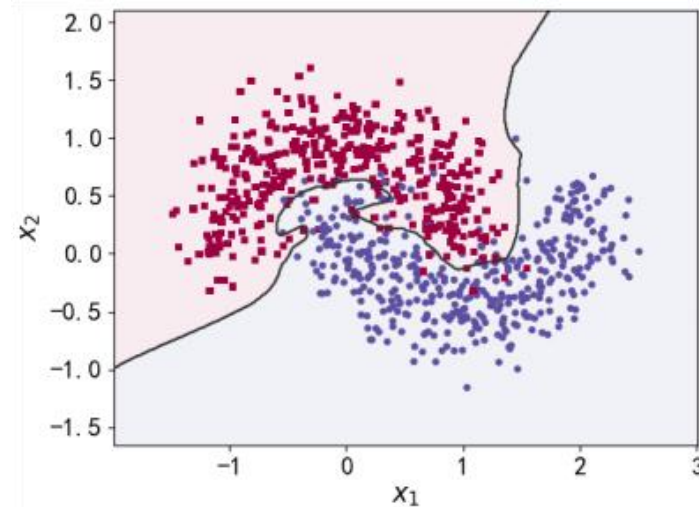
(b) 应用dropout之后的网络

对于神经网络单元，按照一定的概率（20%？50%？）将其暂时从网络中丢弃。

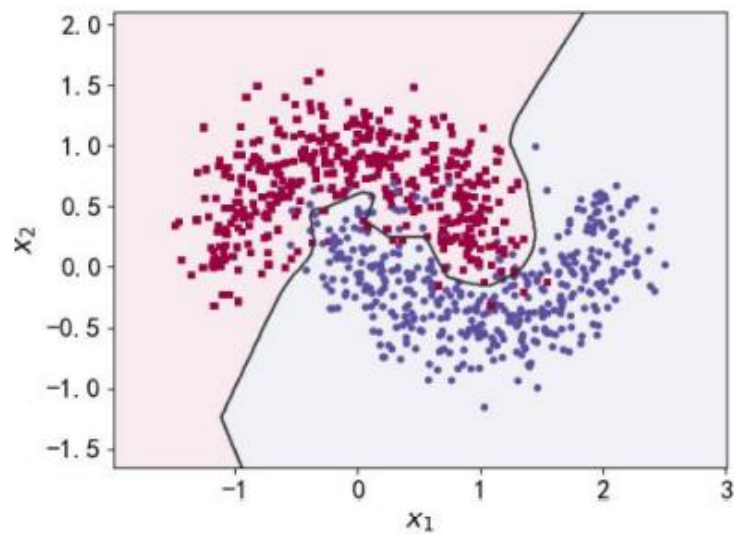
不工作的那些节点可以暂时认为不是网络结构的一部分，但是它的权重得保留下来。



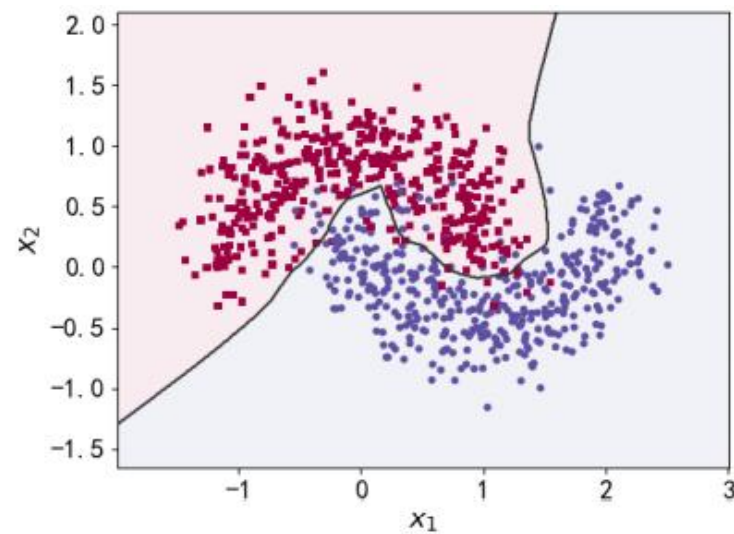
无 Dropout 层



1 层 Dropout 层



2 层 Dropout



4 层 Dropout 层

Dropout

在训练阶段使用，在测试阶段不使用！



构建BP神经网络

```
class MLP(nn.Module):
    def __init__(self):
        super(MLP, self).__init__()
        self.model = nn.Sequential(
            nn.Linear(28*28, 200),
            nn.Dropout(0.5),
            nn.ReLU(inplace=True),
            nn.Linear(200, 200),
            nn.Dropout(0.5),
            nn.ReLU(inplace=True),
            nn.Linear(200, 10),
        )
    def forward(self, x):
        x = self.model(x)
        return x
```

- 训练时，Dropout层以指定的丢弃概率随机丢弃上一层的输出元素；
- 验证 / 测试时（`model.eval()`后），Dropout层不发挥作用。

```
model = MLP()
```

```
optimizer = SGD(model.parameters(), lr=0.01)
```

```
criterion = nn.CrossEntropyLoss() #交叉熵损失
```

```
for step, (b_x, b_y) in enumerate(traindata_loader):
```

```
    b_x, b_y = b_x.to(device), b_y.to(device)
```

```
    if step < train_batch_num: #训练的批次
```

```
        model.train() #设置模型为训练模式
```

```
        output = model(b_x) #前向传播，训练输出
```

```
        loss = criterion(output, b_y) #计算训练损失
```

```
        optimizer.zero_grad()
```

```
        loss.backward() #反向传播，计算参数梯度
```

```
        optimizer.step() #梯度下降优化，调整参数
```

```
    else: #验证的批次
```

```
        model.eval() #设置模型为评估模式
```

```
        output = model(b_x) #验证输出
```

```
        loss = criterion(output, b_y) #验证损失
```

以下哪些方法可以用来防止过拟合？

- ☒ A 正则化
- ☒ B Dropout
- ☒ C 数据增强
- ☒ D 提前终止训练

提交

以下哪些方法可以用来防止过拟合？

- ☐ A 正则化
- ☐ B Dropout
- ☐ C 数据增强
- ☐ D 提前终止训练

提交

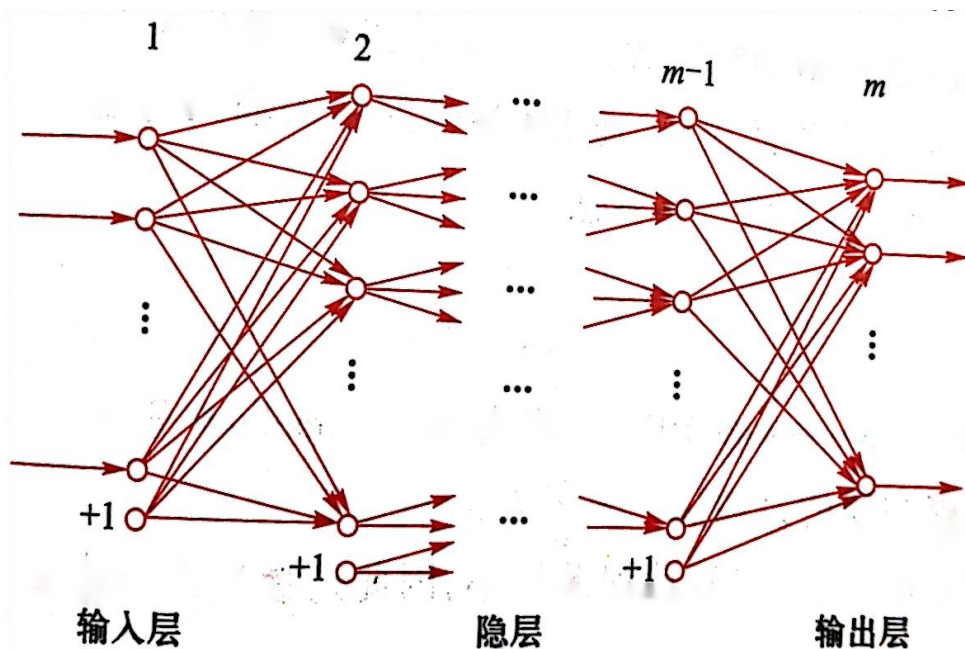
(1) 隐层数、隐层神经元数、激活函数、**学习率**等确定：目前尚无理论指导。

- 网络结构的选择

多层BP网络可用来逼近几乎任一个函数，只要在隐层中有足够的神经元。

但通常并不能说，多少层或多少神经元就足以得到足够的性能。

- 网络结构的选择及防过拟合策略（提前终止训练、L2正则、Dropout等）



□ 使用指数衰减的学习率 lr

$$W_{t+1} = W_t - lr \times \nabla_{W=W_t} J(W)$$

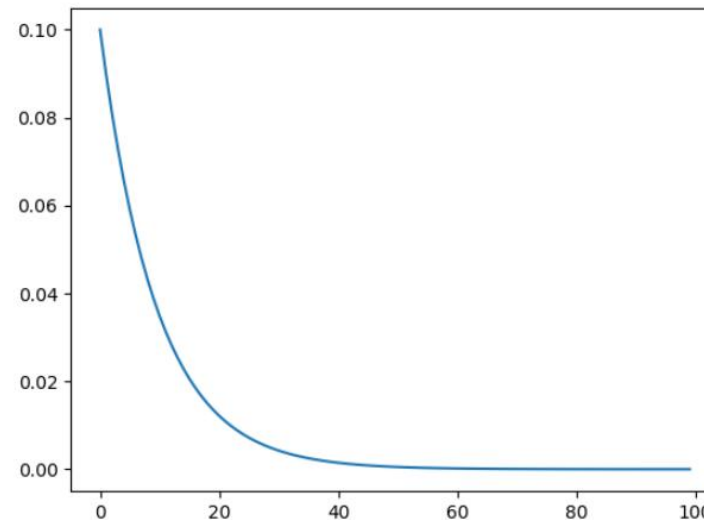
初始学习率

参数
梯度

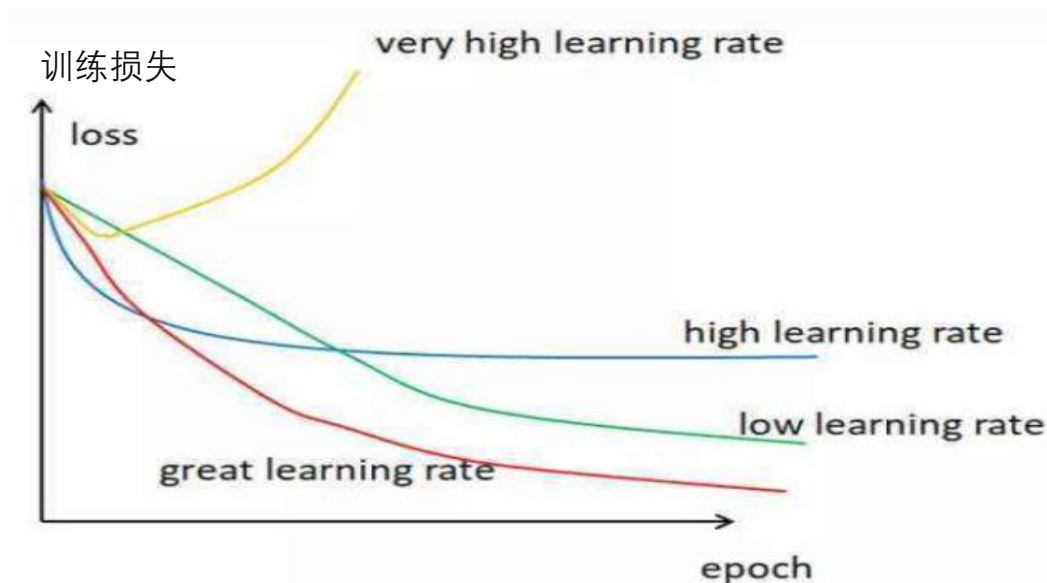
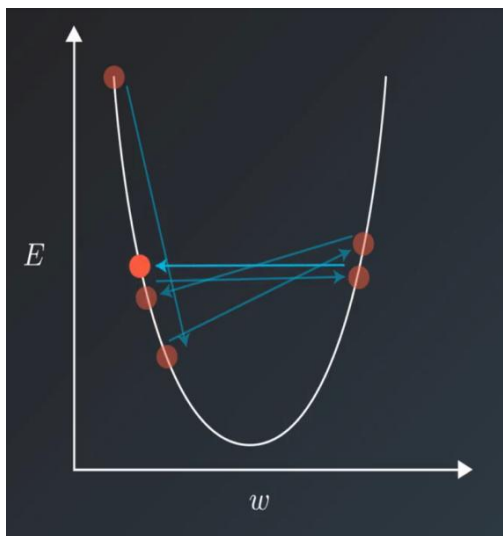
$$lr = lr \times \text{gamma}^{\text{epoch}}$$

学习率调整倍数的底数，
默认为0.1

指数为训练周期
epoch



`torch.optim.lr_scheduler.ExponentialLR(optimizer, gamma=0.1, last_epoch=-1)`



学习率状态	对训练损失的影响	模型训练结果
过小	平滑但缓慢地下降	收敛慢，可能陷入局部最优
合适	稳定且快速地下降	良好收敛，找到较优点
过大	剧烈震荡或下降停滞	无法收敛，模型不稳定
非常大	急剧上升至 NaN	训练迅速失败

□ 使用固定步长衰减的学习率 lr

a) 等间隔调整学习率：每训练 $step_size$ 个 epoch，学习率调整为 $lr = lr * \gamma$

```
torch.optim.lr_scheduler.StepLR(optimizer, step_size, gamma=0.1, last_epoch=-1) #等间隔调整  $lr$ 
```

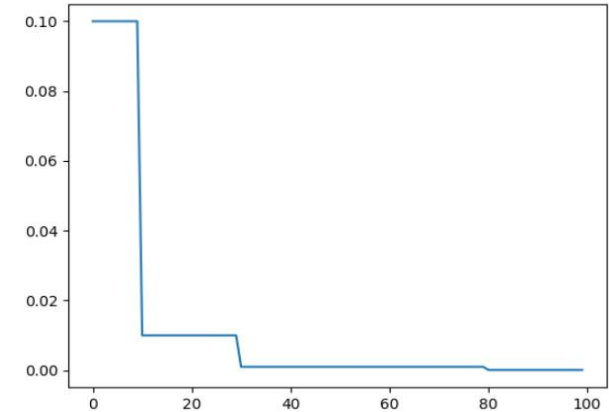
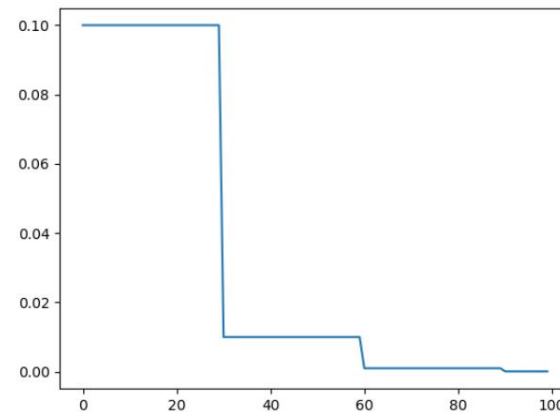
optimizer: 封装的优化器

$step_size$ (int): 学习率下降间隔数，单位是 epoch

γ (float): 学习率调整倍数，默认为 0.1

$last_epoch$: 上一个 epoch 数，指示学习率是否需要调整，默认为 -1

```
# Assuming optimizer uses lr=0.1 for all groups
# lr=0.1   if epoch < 30
# lr=0.01  if 30 <= epoch < 60
# lr=0.001 if 60 <= epoch < 90
# ...
scheduler = StepLR(optimizer, step_size=30, gamma=0.1)
for epoch in range(100):
    scheduler.step()
    train(...)
    validate(...)
```



b) 多间隔调整学习率：学习率调整的间隔并不相等

```
torch.optim.lr_scheduler.MultiStepLR(optimizer, milestones=[10,30,80], gamma=0.1, last_epoch=-1)
```

#按照设定的间隔调整 lr

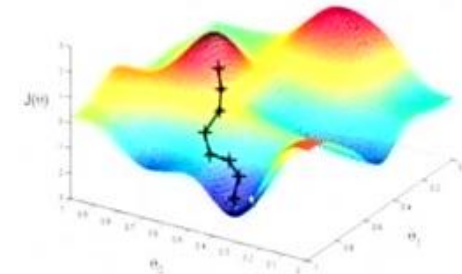
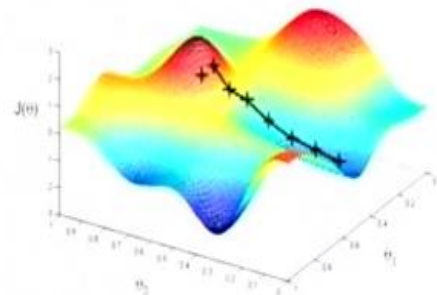
(2) 神经网络参数的初始设置:

能不能初始化为全0? 或者同样的值? 为什么?

全连接神经网络模型:

$$y^k = f^k(W^k y^{k-1} + b^k), \quad k=2, \dots, m$$

梯度爆炸或消失的问题



初始权值常用设置方法: 均匀分布、正态 (高斯) 分布、Xavier等初始化。

Xavier初始化 (Glorot, X. & Bengio, Y., 2010, 适合tanh/sigmoid): 保持前后向方差一致, 减少梯度消失, 加快收敛

`torch.nn.init.xavier_normal_(tensor, gain=1)` 正态分布 $\sim N(0, std)$

其中std的计算公式:

tensor: 需要初始化的参数张量

gain: 可选的放缩因子

$$std = gain \times \sqrt{\frac{2}{fan_in + fan_out}}$$

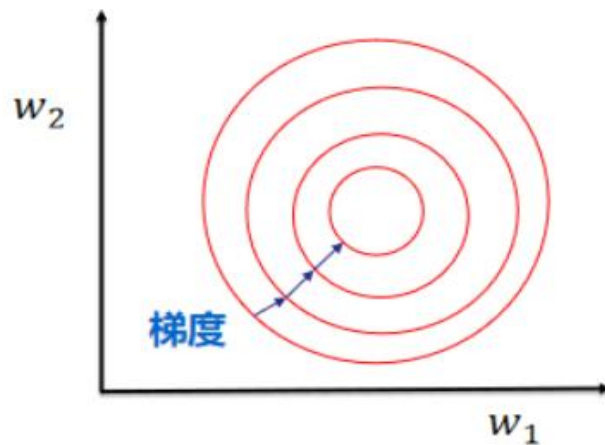
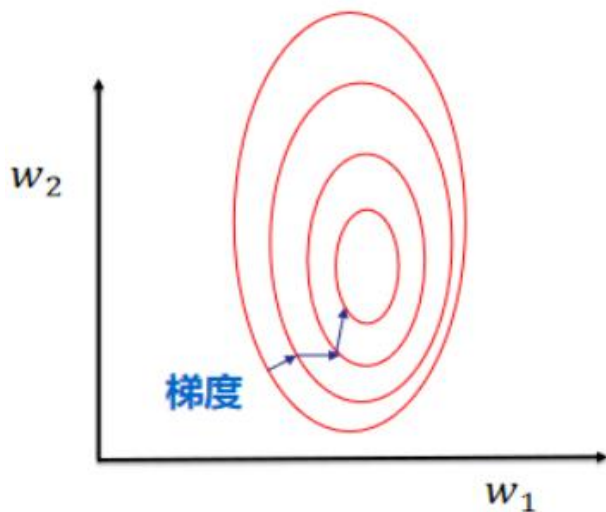
CSDN @ 源仔

特性	Xavier/Glorot	He	LeCun
提出时间	2010	2015	1990s
核心目标	保持前后向方差一致	适应ReLU的置零特性	保持前向方差为1
适用激活函数	tanh, sigmoid	ReLU及其变种	SELU
方差缩放	$\frac{2}{n_{in} + n_{out}}$	$\frac{2}{n_{in}}$	$\frac{1}{n_{in}}$

- (3) **训练数据预处理**：一般作**标准化/归一化**处理，即变换到 $[0,1]$ 或 $[-1,1]$ 区间内，使得在每个训练集上，每个特征的均值为0，且具有相同的方差。

提升模型精度：不同维度之间的特征在数值上有一定比较性，可以大大提高分类器的准确性。

加速模型收敛：最优解的寻优过程明显会变得平缓，更容易正确的收敛到最优解。



图像：归一化 + 数据增强
文本：分词 + 去停用词 + 编码（如词嵌入）
时间序列：差分 + 标准化

- (4) **后处理过程**：根据任务需求对模型输出进行优化和转换，例如二分类任务阈值筛选（Thresholding）、多分类任务Top-K 筛选、类别修正、概率校准（Probability Calibration）等

`torch.optim.SGD`: 随机梯度下降

`torch.optim.ASGD`: 平均SGD

`torch.optim.Rprop`: 弹性反向传播

`torch.optim.Adagrad`: 自适应梯度算法

`torch.optim.Adadelta`: Adadelta算法

`torch.optim.RMSprop`: 均方根传播

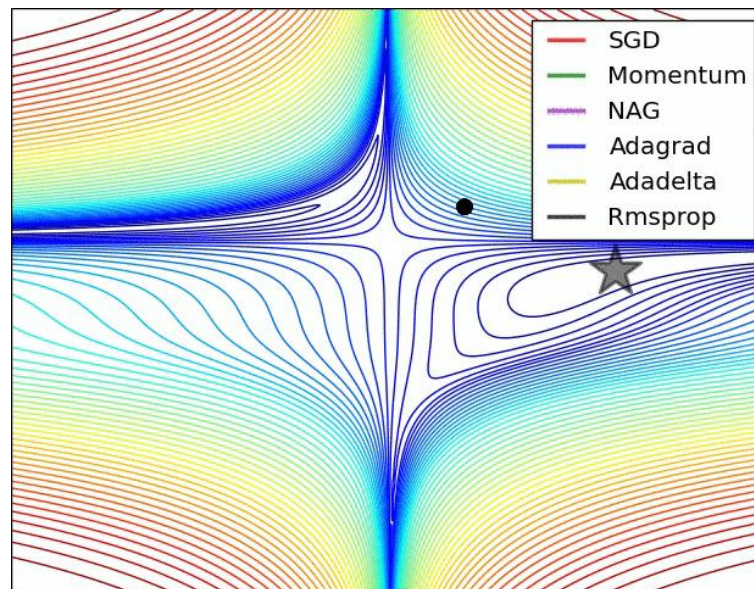
`torch.optim.Adam`: Adam算法

`torch.optim.Adamax`: Adam的无限范数变种

...

```
optimizer = optim.SGD(net.parameters(), lr=0.01, momentum=0.9)
```

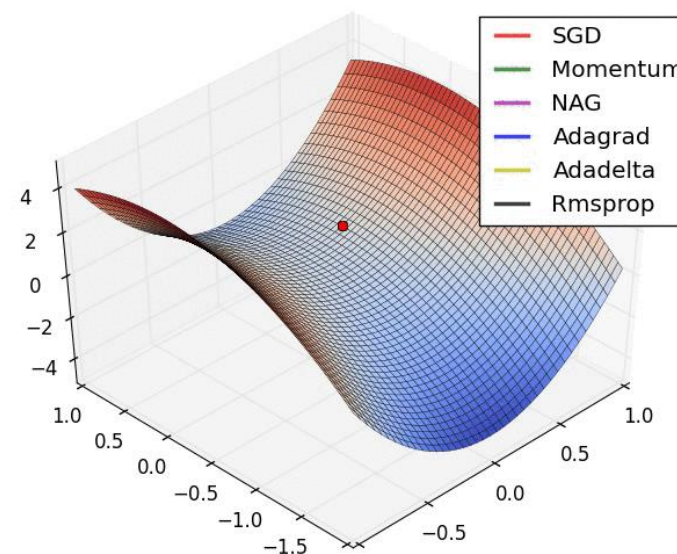
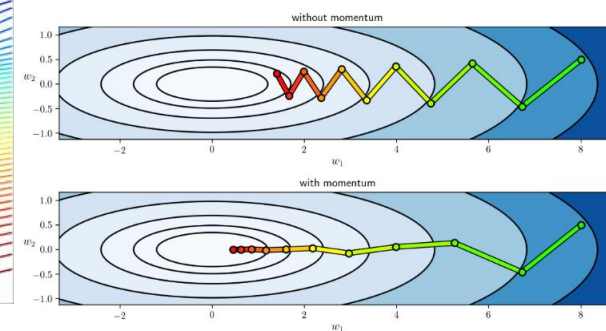
动量 (Momentum) 是SGD及其变种 (如ASGD) 的一个可选参数, 而自适应学习率方法 (如Adagrad、RMSprop、Adam等) 通常内置了动量的概念或类似机制。



Momentum算法:

$$m_t = \gamma \times m_{t-1} + lr \times \nabla_{W=W_t} J(W)$$

$$W_{t+1} = W_t - m_t$$



`torch.optim.SGD`: 随机梯度下降

`torch.optim.ASGD`: 平均SGD

`torch.optim.Rprop`: 弹性反向传播

`torch.optim.Adagrad`: 自适应梯度算法

`torch.optim.Adadelta`: Adadelta算法

`torch.optim.RMSprop`: 均方根传播

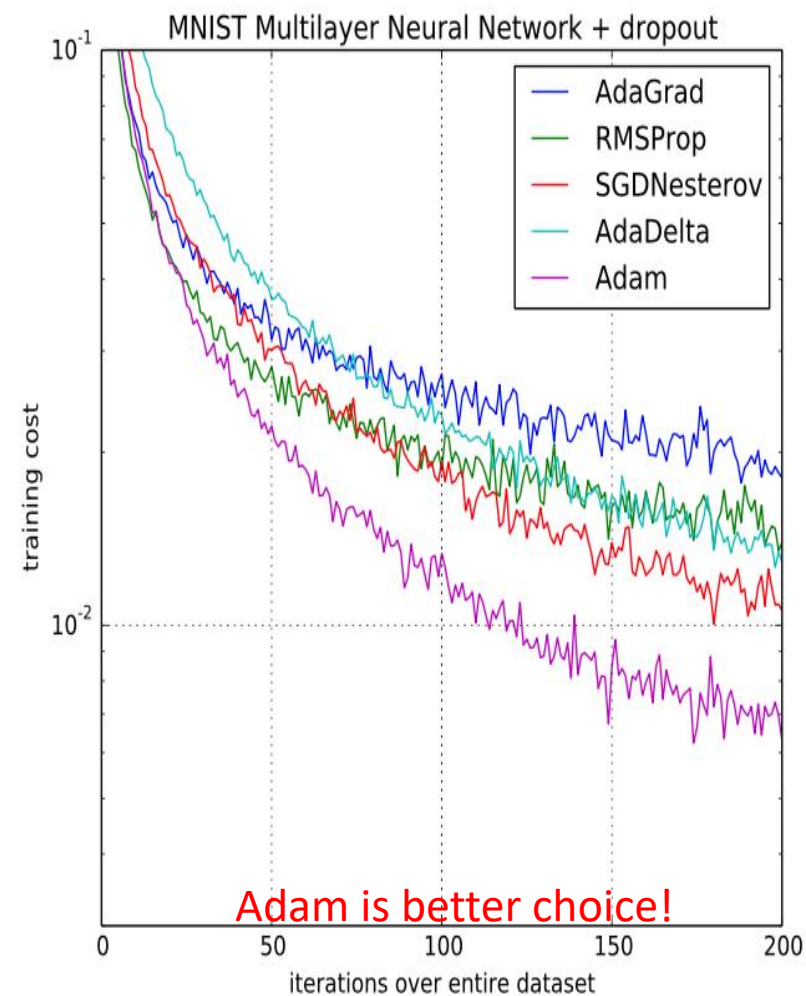
`torch.optim.Adam`: Adam算法

`torch.optim.Adamax`: Adam的无限范数变种

```
optimizer = optim.SGD(net.parameters(), lr=0.01, momentum=0.9)
```

```
optimizer = Adam(net.parameters(), lr=0.01)
```

ADAM: a method for stochastic optimization
[Kingma et al. 2015]



<https://ruder.io/optimizing-gradient-descent/>

http://www.360doc.com/content/21/0126/16/73546223_959051808.shtml

优化器	核心原理	优点	缺点	适用场景
SGD （随机梯度下降）	随机选取样本计算梯度，更新公式： $\mathbf{w} = \mathbf{w} - \text{lr} * \nabla \mathbf{w}$	简单直观，内存占用低，适合大规模数据 / 模型；容易逃离局部最优	学习率固定，需手动调参；收敛慢，易在鞍点震荡	基础 baseline、需要精细调参的场景；大型 CNN/RNN（如训练 ResNet）
ASGD	SGD 的改进：对后期参数做平均（类似滑动平均），降低波动	收敛更稳定，泛化能力略强于 SGD	计算量略高于 SGD；需要存储历史参数平均值	替代 SGD，希望在收敛稳定性上提升一点点的场景
Rprop （弹性反向传播）	仅用梯度符号更新（去掉梯度大小），自适应调整每个参数的学习率	对梯度尺度不敏感，适合处理稀疏梯度（如文本任务）	不适合噪声大的数据；学习率调整可能过度依赖历史符号，稳定性一般	早期用于反向传播，现在较少用，可尝试用于稀疏特征场景（如 NLP）
Adagrad	对每个参数的学习率除以历史梯度平方和的平方根（累积梯度平方）	自动降低高频特征的学习率，适合稀疏数据（如文本、推荐系统）	学习率随迭代单调递减，后期可能因学习率过小导致收敛停滞	稀疏特征任务（如点击率预测、NLP）；不适合长训练周期任务
Adadelta	改进 Adagrad：用滑动窗口替代累积梯度平方，避免学习率衰减到 0	无需手动设置初始学习率；学习率更稳定，适合长周期训练	对超参数（窗口大小）较敏感	替代 Adagrad，需要长训练周期的任务（如语音识别）
RMSprop （均方根传播）	类似 Adadelta，用指数移动平均替代梯度平方，动态调整学习率	收敛速度快于 SGD，解决 Adagrad 学习率衰减问题；稳定性好	仍需手动设置初始学习率；可能收敛到局部最优	广泛用于递归神经网络（RNN/LSTM）；图像生成（如 GAN）
Adam （自适应矩估计）	结合动量（Momentum）和 RMSprop：用一阶矩（动量）和二阶矩（自适应学习率）	收敛快，稳定性强，对超参数不敏感；几乎适用于所有场景	可能在某些任务中泛化能力略逊于 SGD（需配合学习率调度）	绝大多数深度学习任务的默认选择（图像分类、目标检测、NLP、推荐等）
Adamax	Adam 的变种：用无穷范数替代二阶矩的平方根，学习率更新更稳健	对异常值更鲁棒，学习率衰减更慢，适合噪声数据	收敛速度略慢于 Adam；在部分场景下效果与 Adam 接近	替代 Adam，处理含噪声或异常值较多的数据（如传感器数据、医疗影像）



torch.optim.SGD: 随机梯度下降

torch.optim.ASGD: 平均SGD

torch.optim.Rprop: 弹性反向传播

torch.optim.Adagrad: 自适应梯度算法

torch.optim.Adadelta: Adadelta算法

torch.optim.RMSprop: 均方根传播

torch.optim.Adam: Adam算法

torch.optim.Adamax: Adam的无限范数变种

```
optimizer = optim.SGD(net.parameters(), lr=0.01, momentum=0.9)
```

```
optimizer = Adam(net.parameters(), lr=0.01)
```

<https://ruder.io/optimizing-gradient-descent/>

http://www.360doc.com/content/21/0126/16/73546223_959051808.shtml

● 选择依据

1.收敛速度和稳定性: 如果需要快速收敛且希望算法稳定, *Adam*是较好的选择, 它覆盖 90% 以上的场景 (除非有特定需求)。

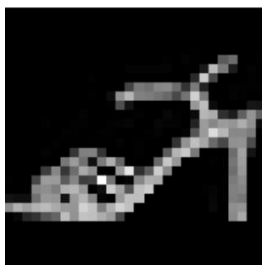
2.数据特性: 对于稀疏数据, *Adamax*表现更好; 对于一般数据, *Adam*和*SGD with Momentum*都是不错的选择。

3.计算资源: 如果计算资源有限, *SGD with Momentum*可能更合适, 因为它不需要存储历史的平方梯度信息。

对于大多数应用, 从*Adam*开始, 如果训练不稳定或性能不佳, 再尝试*SGD+Momentum*或其他优化器。

Fashion-MNIST 数据集是德国 Zalando 公司提供的衣物图像数据集，包含 60,000 个样本的训练集和 10,000 个样本的测试集。

每个样本都是 28x28 灰度图像，与 10 个类别的标签相关联。Fashion-MNIST 数据集旨在作为原始 MNIST 数据集的直接替代品，用于对机器学习算法进行基准测试。



Sandals

Label	Description	Examples
0	T-Shirt/Top	
1	Trouser	
2	Pullover	
3	Dress	
4	Coat	
5	Sandals	
6	Shirt	
7	Sneaker	
8	Bag	
9	Ankle boots	



8.1 若对手写的大写英文字母（A~Z）图片进行分类，训练集中每张图片为灰度图，其大小为14*14像素，假设采用含有2个隐层（隐层神经元个数分别为300和100）的BP神经网络，试问要训练的神经网络参数量为多少？并选择输出层的激活函数及训练的代价函数。
（提示：输入为各个像素点的灰度值，输出采用one-hot编码）

8.2 用含有1个隐层的BP网络拟合如下的非线性函数：

$$f(x_1, x_2) = 100(x_1^2 - x_2)^2 + (1 - x_1)^2$$
$$-2.048 \leq x_i \leq 2.048 \quad i = 1, 2$$

- 1) 选择隐层和输出神经元的激活函数，并给出训练的代价函数。
- 2) 假设隐层神经元有5个神经元，计算该BP网络的参数量和计算量（FLOPS，乘加运算次数），给出一个训练样本示例，并说明如果所设计的BP网络能够被较好地推广，即具有较好的泛化能力，一般情况下至少需要多少个训练样本？



- 1) 如何衡量分类器对数据的误分类程度？理解损失函数、代价（成本）函数和目标函数的基本概念。
- 2) 如何利用误分类数据来调整分类器的参数？理解梯度下降法的基本概念。
- 3) 深度学习中的深度指什么？深度学习的三个步骤是什么？深度学习有哪些主要的优化算法，如何根据实际情况选择优化算法？
- 4) 如何根据具体的问题设计**BP**神经网络结构及数学模型描述？**BP**神经网络的结构特点是什么？如何计算全连接层的参数量和计算量？
- 5) **BP**神经网络是怎么训练的，神经网络训练的三个主要步骤是什么？批量大小对神经网络训练有什么影响？解决梯度消失的方法有哪些？学习率对随机梯度下降法有什么影响，调整学习率的策略有哪些？
- 6) 什么是泛化能力？解决神经网络过拟合问题的方法有哪些？
- 7) 数据预处理为什么要标准化/归一化？